modeling_with_cleaned_data

Quynh Tran

2026-04-10

Import packages

```r
library(MASS)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.5.2
## v ggplot2   4.0.0     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## x dplyr::select() masks MASS::select()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
#install.packages("lattice")
library(lattice)
library(caret)
```

```
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
#install.packages("Matrix")
library(Matrix)
```

```
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```r
library(glmnet)
```

```
## Loaded glmnet 4.1-10
```

```r
library(mlbench)
```

```
## Warning: package 'mlbench' was built under R version 4.5.3
```

```r
library(caret)
library(here)
```

```
## Warning: package 'here' was built under R version 4.5.3
```

```
## here() starts at C:/Users/roddu/Downloads/glassdoor-company-reviews
```

Import dataset

```r
df <- read.csv(here("data/processed/reviews_with_sent_scores.csv"))
#view(df)
#summary(df)
```

Logistic Regression model:

Create a subset for Logistic regression and Convert categorical variables:

```r
log_data <- df %>%
  select(
    flags_recommend_frend,
    rating_culture_values,
    rating_work_life,
    rating_compensation_benefits,
    rating_senior_leadership,
    rating_career_opportunities,
    rating_diversity_inclusion,
    flags_business_outlook,
    flags_ceo_approval,
    employee_length,
    employee_job_quit,
    sector,
    covid_related_review,
    career_missing
  )
```

```r
# Recode recommendation / outlook / CEO approval into numeric variables
log_data <- log_data %>%
  dplyr::mutate(
    employee_job_quit = as.integer(employee_job_quit),
    career_missing = as.integer(career_missing),
    flags_recommend_frend = as.factor(flags_recommend_frend),
    sector = as.integer(as.factor(sector)),
    flags_business_outlook = as.integer(as.factor(flags_business_outlook)),
    flags_ceo_approval = as.integer(as.factor(flags_ceo_approval))
  )
```

Split the data into training-testing

```r
library(caret)

set.seed(123)

train_index <- createDataPartition(
  log_data$flags_recommend_frend,
  p = 0.8,
  list = FALSE
)

train.data <- log_data[train_index, ]
test.data <- log_data[-train_index, ]

# Check class imbalance after splitting
prop.table(table(train.data$flags_recommend_frend))
```

```
##
##  NEGATIVE  POSITIVE
## 0.6916355 0.3083645
```

```r
prop.table(table(test.data$flags_recommend_frend))
```

```
##
##  NEGATIVE  POSITIVE
## 0.6934673 0.3065327
```

```r
# Fit the model
set.seed(123)
model_logit <- glm(
  flags_recommend_frend ~.,
  data = train.data,
  family = binomial
)

summary(model_logit)
```

```
##
## Call:
## glm(formula = flags_recommend_frend ~ ., family = binomial, data = train.data)
##
## Coefficients:
##                              Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  -7.84008    1.17543  -6.670 2.56e-11 ***
## rating_culture_values         0.62444    0.15347   4.069 4.72e-05 ***
## rating_work_life              0.28724    0.12595   2.281 0.022574 *
## rating_compensation_benefits  0.07409    0.13581   0.546 0.585356
## rating_senior_leadership      0.49393    0.15378   3.212 0.001318 **
## rating_career_opportunities   0.58309    0.19915   2.928 0.003413 **
## rating_diversity_inclusion    0.01223    0.14054   0.087 0.930659
## flags_business_outlook        1.44406    0.21039   6.864 6.71e-12 ***
## flags_ceo_approval           -0.11902    0.16659  -0.714 0.474939
## employee_length               0.02116    0.03227   0.656 0.511957
## employee_job_quit            -0.78753    0.57994  -1.358 0.174475
## sector                       -0.04566    0.02945  -1.550 0.121111
## covid_related_review         -0.07563    0.34217  -0.221 0.825074
## career_missing               -0.98802    0.29109  -3.394 0.000688 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 989.69  on 800  degrees of freedom
## Residual deviance: 340.61  on 787  degrees of freedom
## AIC: 368.61
##
## Number of Fisher Scoring iterations: 6
```

```r
# Predict on test data
log_pred_prob <- predict(model_logit, test.data, type = "response")
#log_pred_prob
```

Evaluate different classification thresholds to select the best one using F1 scores

F1 = 2 * (precision * recall) / (precision + recall)

```r
# Step 5.1: Create a Sequence of Thresholds
thresholds <- seq(0.1, 1.0, by = 0.1)

# Step 5.2: Define a function to calculate F1 score
f1_score <- function(actual, predicted) {

  tp <- sum(actual == "POSITIVE" & predicted == "POSITIVE")
  fp <- sum(actual == "NEGATIVE" & predicted == "POSITIVE")
  fn <- sum(actual == "POSITIVE" & predicted == "NEGATIVE")
```

```r
  precision <- ifelse((tp + fp) == 0, 0, tp / (tp + fp))
  recall <- ifelse((tp + fn) == 0, 0, tp / (tp + fn))

  if (precision + recall == 0) return(0)

  f1 <- 2 * precision * recall / (precision + recall)

  return(f1)
}

# Step 5.3: Loop Through Thresholds
f1_results <- data.frame(threshold = thresholds, F1 = NA)

for (i in seq_along(thresholds)) {

  t <- thresholds[i]

  pred_class <- ifelse(log_pred_prob > t, "POSITIVE", "NEGATIVE")

  f1_results$F1[i] <- f1_score(test.data$flags_recommend_frend, pred_class)
}

# Step 5.4: Print F1 scores
print(f1_results)
```

```
##    threshold        F1
## 1        0.1 0.7189542
## 2        0.2 0.7611940
## 3        0.3 0.8032787
## 4        0.4 0.8135593
## 5        0.5 0.7747748
## 6        0.6 0.7747748
## 7        0.7 0.7358491
## 8        0.8 0.7142857
## 9        0.9 0.6666667
## 10       1.0 0.0000000
```

```r
# Step 5.5: Choose the threshold with the best F1 score
best_threshold <- f1_results$threshold[which.max(f1_results$F1)]
best_f1 <- max(f1_results$F1)

best_threshold
```

```
## [1] 0.4
```

```r
best_f1
```

```
## [1] 0.8135593
```

Convert Probabilities to Class Predictions using threshold of 0.4

```
log_pred_class <- ifelse(log_pred_prob > 0.4,"POSITIVE", "NEGATIVE")

# Convert to factor
log_pred_class <- as.factor(log_pred_class)
```

Evaluate model performance

```
logistic_cm <- confusionMatrix(log_pred_class, test.data$flags_recommend_frend)
#logistic_cm
```

ROC Curve and AUC

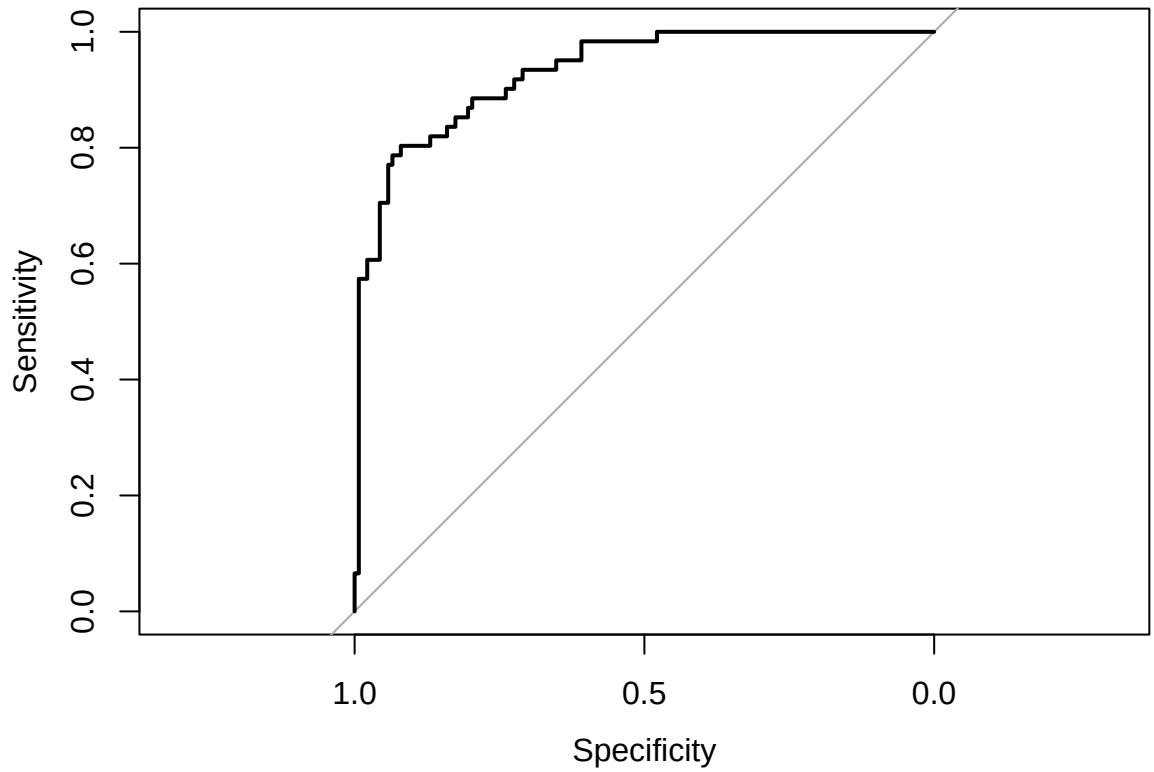
```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.

##
## Attaching package: 'pROC'

## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
log_roc_curve <- roc(test.data$flags_recommend_frend, log_pred_prob)
```

```
## Setting levels: control = NEGATIVE, case = POSITIVE

## Setting direction: controls < cases
```

```
plot(log_roc_curve)
```




```
auc(log_roc_curve)
```

```
## Area under the curve: 0.9304
```

Function to evaluate each regularized model (Lasso, Ridge, Elastic Net)

```
evaluate_model <- function(prob, actual, model_name, thresholds = seq(0.1, 1.0, by = 0.05)) {

  f1_results <- data.frame(threshold = thresholds, F1 = NA)

  for (i in seq_along(thresholds)) {
    t <- thresholds[i]
    pred_class <- ifelse(prob > t, "POSITIVE", "NEGATIVE")
    f1_results$F1[i] <- f1_score(actual, pred_class)
  }

  best_threshold <- f1_results$threshold[which.max(f1_results$F1)]
  cat("Best threshold for", model_name, ":", best_threshold, "\n")

  pred_class <- ifelse(prob > best_threshold, "POSITIVE", "NEGATIVE")
  pred_class <- as.factor(pred_class)
```

```r
  cm <- confusionMatrix(pred_class, actual)
  print(cm)

  roc_obj <- roc(actual, prob)
  plot(roc_obj, main = paste("ROC Curve -", model_name))

  model_auc <- auc(roc_obj)
  cat(model_name, "AUC:", model_auc, "\n")

  list(
    f1_results = f1_results,
    best_threshold = best_threshold,
    pred_class = pred_class,
    confusion_matrix = cm,
    roc = roc_obj,
    auc = model_auc
  )
}
```

Lasso

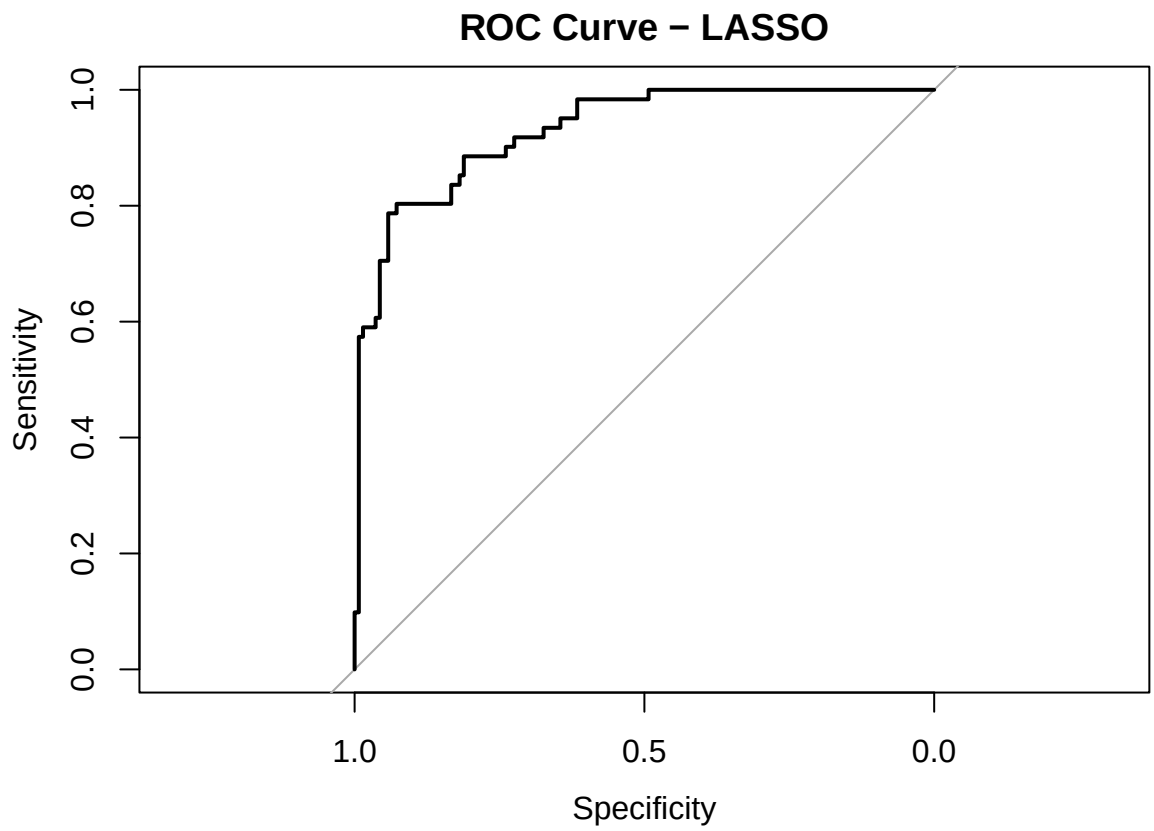
```r
x_train <- model.matrix(flags_recommend_frend ~ ., data = train.data)[, -1]
x_test  <- model.matrix(flags_recommend_frend ~ ., data = test.data)[, -1]

y_train <- ifelse(train.data$flags_recommend_frend == "POSITIVE", 1, 0)

cv_lasso <- cv.glmnet(
  x_train,
  y_train,
  family = "binomial",
  alpha = 1
)

lasso_prob <- predict(
  cv_lasso,
  newx = x_test,
  s = "lambda.min",
  type = "response"
)

lasso_prob <- as.numeric(lasso_prob)

lasso_eval <- evaluate_model(
  prob = lasso_prob,
  actual = test.data$flags_recommend_frend,
  model_name = "LASSO"
)
```

```
## Best threshold for LASSO : 0.45
## Confusion Matrix and Statistics
##
```

```
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      130       13
##   POSITIVE        8       48
##
##                Accuracy : 0.8945
##                  95% CI : (0.8432, 0.9335)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.388e-11
##
##                   Kappa : 0.746
##
##  Mcnemar's Test P-Value : 0.3827
##
##             Sensitivity : 0.9420
##             Specificity : 0.7869
##          Pos Pred Value : 0.9091
##          Neg Pred Value : 0.8571
##              Prevalence : 0.6935
##          Detection Rate : 0.6533
##    Detection Prevalence : 0.7186
##       Balanced Accuracy : 0.8645
##
##        'Positive' Class : NEGATIVE
##
```

```
## Setting levels: control = NEGATIVE, case = POSITIVE
```

```
## Setting direction: controls < cases
```




```
## LASSO AUC: 0.9300309
```

Ridge

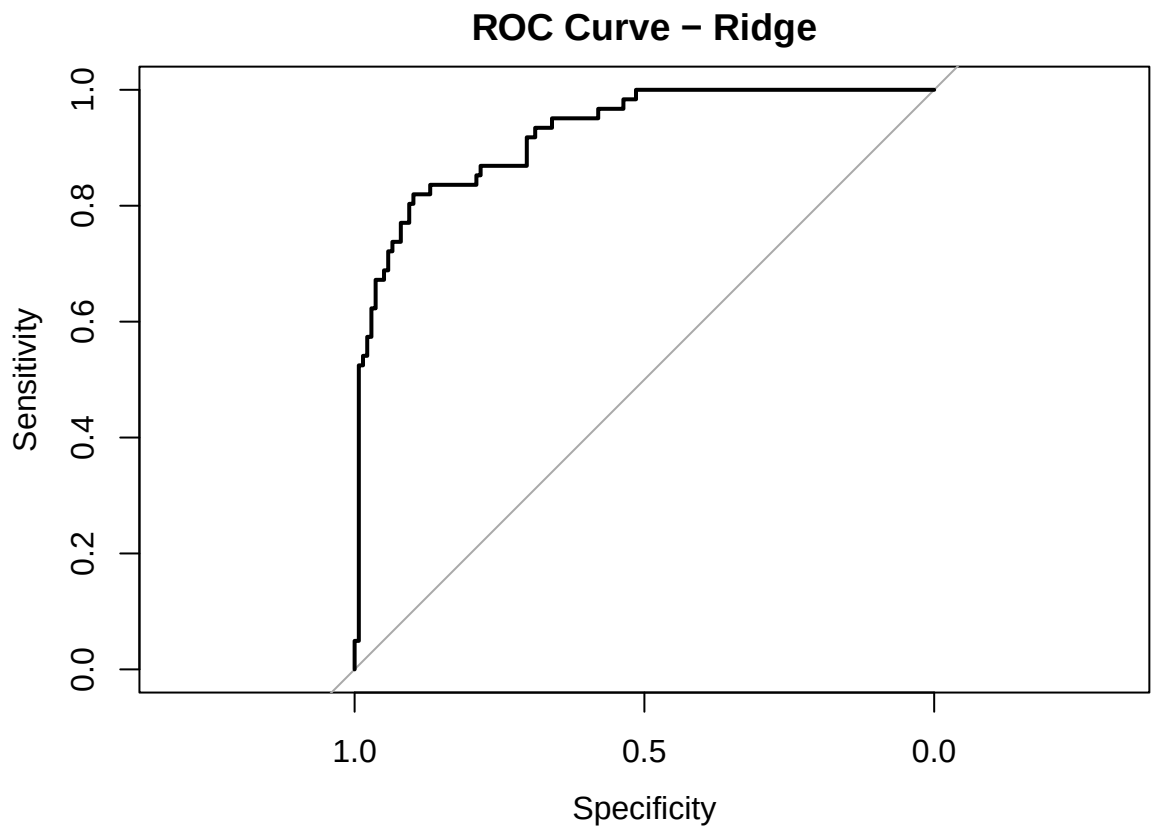
```r
set.seed(123)
cv_ridge <- cv.glmnet(
  x_train, y_train,
  family = "binomial",
  alpha = 0
)

ridge_prob <- predict(cv_ridge, newx = x_test, s = "lambda.min", type = "response")
ridge_prob <- as.numeric(ridge_prob)

ridge_eval <- evaluate_model(
  prob = ridge_prob,
  actual = test.data$flags_recommend_frend,
  model_name = "Ridge"
)
```

```
## Best threshold for Ridge : 0.35
## Confusion Matrix and Statistics
##
```

```
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      124       11
##   POSITIVE       14       50
##
##                Accuracy : 0.8744
##                  95% CI : (0.8202, 0.917)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.833e-09
##
##                   Kappa : 0.7085
##
##  Mcnemar's Test P-Value : 0.6892
##
##             Sensitivity : 0.8986
##             Specificity : 0.8197
##          Pos Pred Value : 0.9185
##          Neg Pred Value : 0.7812
##              Prevalence : 0.6935
##          Detection Rate : 0.6231
##    Detection Prevalence : 0.6784
##       Balanced Accuracy : 0.8591
##
##        'Positive' Class : NEGATIVE
##
```

```
## Setting levels: control = NEGATIVE, case = POSITIVE
```

```
## Setting direction: controls < cases
```




```
## Ridge AUC: 0.9243288
```

Elastic Net

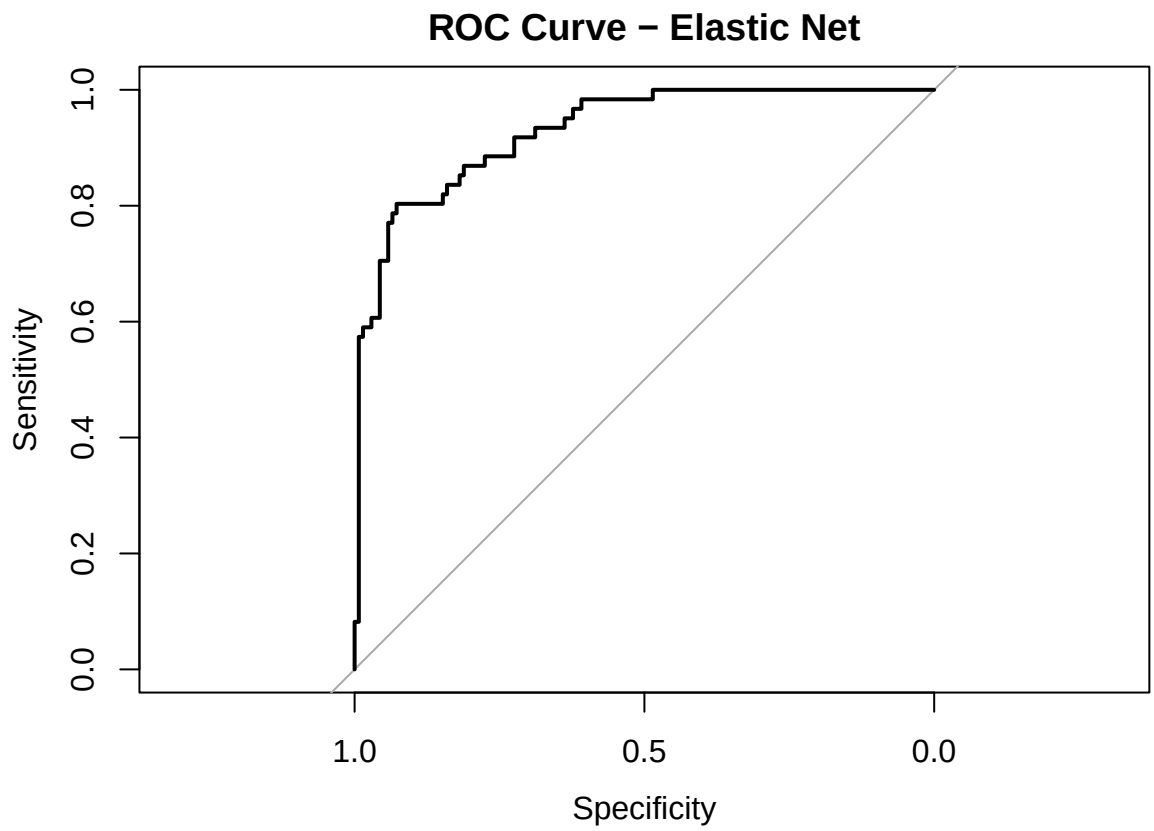
```r
set.seed(123)
cv_enet <- cv.glmnet(
  x_train, y_train,
  family = "binomial",
  alpha = 0.5
)

enet_prob <- predict(cv_enet, newx = x_test, s = "lambda.min", type = "response")
enet_prob <- as.numeric(enet_prob)

enet_eval <- evaluate_model(
  prob = enet_prob,
  actual = test.data$flags_recommend_frend,
  model_name = "Elastic Net"
)
```

```
## Best threshold for Elastic Net : 0.35
## Confusion Matrix and Statistics
##
```

```
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      128       12
##   POSITIVE       10       49
##
##                Accuracy : 0.8894
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7376
##
##  Mcnemar's Test P-Value : 0.8312
##
##             Sensitivity : 0.9275
##             Specificity : 0.8033
##          Pos Pred Value : 0.9143
##          Neg Pred Value : 0.8305
##              Prevalence : 0.6935
##          Detection Rate : 0.6432
##    Detection Prevalence : 0.7035
##       Balanced Accuracy : 0.8654
##
##        'Positive' Class : NEGATIVE
##
```

```
## Setting levels: control = NEGATIVE, case = POSITIVE
```

```
## Setting direction: controls < cases
```




```
## Elastic Net AUC: 0.9294369
```

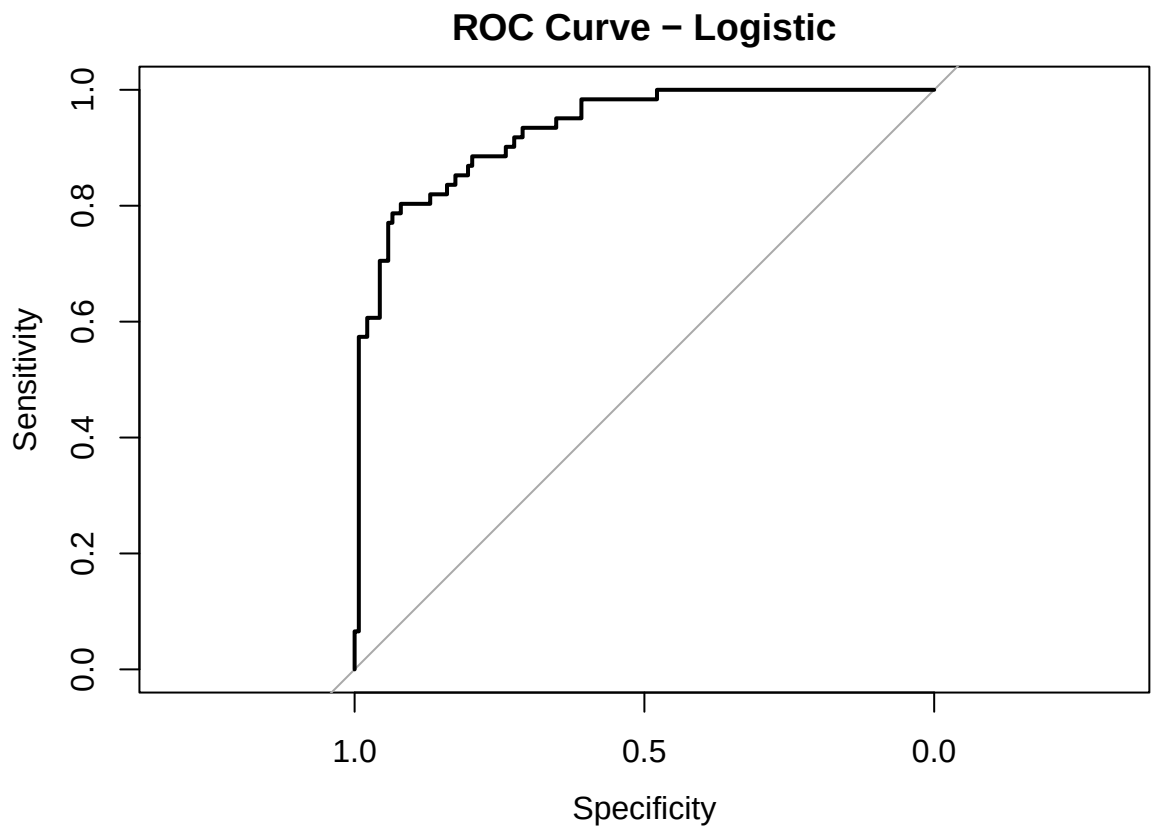
Logit again

```r
set.seed(123)

model_logit <- glm(
  flags_recommend_frend ~ .,
  data = train.data,
  family = binomial
)

logit_prob <- predict(
  model_logit,
  newdata = test.data,
  type = "response"
)

logit_eval <- evaluate_model(
  prob = log_pred_prob,
  actual = test.data$flags_recommend_frend,
  model_name = "Logistic"
)
```

```
## Best threshold for Logistic : 0.4
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      129       13
##   POSITIVE        9       48
##
##                Accuracy : 0.8894
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7351
##
##  Mcnemar's Test P-Value : 0.5224
##
##             Sensitivity : 0.9348
##             Specificity : 0.7869
##          Pos Pred Value : 0.9085
##          Neg Pred Value : 0.8421
##              Prevalence : 0.6935
##          Detection Rate : 0.6482
##    Detection Prevalence : 0.7136
##       Balanced Accuracy : 0.8608
##
##        'Positive' Class : NEGATIVE
##
```

```
## Setting levels: control = NEGATIVE, case = POSITIVE
```

```
## Setting direction: controls < cases
```




```
## Logistic AUC: 0.9303873
```

```
model_comparison <- data.frame(
  Model = c("Logistic", "LASSO", "Ridge", "Elastic Net"),
  Best_Threshold = c(
    logit_eval$best_threshold,
    lasso_eval$best_threshold,
    ridge_eval$best_threshold,
    enet_eval$best_threshold
  ),
  AUC = c(
    as.numeric(logit_eval$auc),
    as.numeric(lasso_eval$auc),
    as.numeric(ridge_eval$auc),
    as.numeric(enet_eval$auc)
  ),
  Accuracy = c(
    as.numeric(logit_eval$confusion_matrix$overall["Accuracy"]),
    as.numeric(lasso_eval$confusion_matrix$overall["Accuracy"]),
    as.numeric(ridge_eval$confusion_matrix$overall["Accuracy"]),
    as.numeric(enet_eval$confusion_matrix$overall["Accuracy"])
  ),
  F1 = c(
    max(logit_eval$f1_results$F1),
    max(lasso_eval$f1_results$F1),
```

```
    max(ridge_eval$f1_results$F1),
    max(enet_eval$f1_results$F1)
  )
)

model_comparison
```

```
##         Model Best_Threshold       AUC  Accuracy        F1
## 1    Logistic           0.40 0.9303873 0.8894472 0.8135593
## 2       LASSO           0.45 0.9300309 0.8944724 0.8205128
## 3       Ridge           0.35 0.9243288 0.8743719 0.8000000
## 4 Elastic Net           0.35 0.9294369 0.8894472 0.8166667
```

Standard Logistic Regression achieves the highest AUC (0.9304), but only marginally above LASSO and Elastic Net, suggesting little gain from regularization in discrimination performance.

In terms of classification performance, LASSO stands out with the highest F1 score (0.8205) and accuracy (0.8945), indicating a slightly better balance between precision and recall. Ridge performs the worst across all metrics, with noticeably lower accuracy and F1, suggesting it may be over-smoothing important features.

Overall, the differences are small, but LASSO provides the best trade-off between predictive accuracy and model simplicity, making it the most reliable choice among the base-only models.

Sentiment Add-Ons

Calculate mean and sds across each model's text columns to get the average or compound sentiment

```
df$sent_vader_avg <- rowMeans(
  df[, c(
    "vader_summary_compound",
    "vader_pros_compound",
    "vader_cons_compound",
    "vader_advice_compound"
  )],
  na.rm = TRUE
)

df$sent_vader_sd <- apply(
  df[, c(
    "vader_summary_compound",
    "vader_pros_compound",
    "vader_cons_compound",
    "vader_advice_compound"
  )],
  1,
  sd,
  na.rm = TRUE
)

df$sent_distilbert_avg <- rowMeans(
```

```
  df[, c(
    "distilbert_summary_signed",
    "distilbert_pros_signed",
    "distilbert_cons_signed",
    "distilbert_advice_signed"
  )],
  na.rm = TRUE
)

df$sent_distilbert_sd <- apply(
  df[, c(
    "distilbert_summary_signed",
    "distilbert_pros_signed",
    "distilbert_cons_signed",
    "distilbert_advice_signed"
  )],
  1,
  sd,
  na.rm = TRUE
)

df$sent_roberta_avg <- rowMeans(
  df[, c(
    "roberta_summary_signed",
    "roberta_pros_signed",
    "roberta_cons_signed",
    "roberta_advice_signed"
  )],
  na.rm = TRUE
)

df$sent_roberta_sd <- apply(
  df[, c(
    "roberta_summary_signed",
    "roberta_pros_signed",
    "roberta_cons_signed",
    "roberta_advice_signed"
  )],
  1,
  sd,
  na.rm = TRUE
)
```

Just column binding the new compounded sentiment scores to original log_data

```
log_data_vader <- cbind(
  log_data,
  df[, c(
    "sent_vader_avg",
```

```r
    "sent_vader_sd"
  )]
)

log_data_distilbert <- cbind(
  log_data,
  df[, c(
    "sent_distilbert_avg",
    "sent_distilbert_sd"
  )]
)

log_data_roberta <- cbind(
  log_data,
  df[, c(
    "sent_roberta_avg",
    "sent_roberta_sd"
  )]
)

log_data_all <- cbind(
  log_data,
  df[, c(
    "sent_vader_avg",
    "sent_vader_sd",
    "sent_distilbert_avg",
    "sent_distilbert_sd",
    "sent_roberta_avg",
    "sent_roberta_sd"
  )]
)

datasets <- list(
  base_only = log_data,
  vader = log_data_vader,
  distilbert = log_data_distilbert,
  roberta = log_data_roberta,
  base_and_all_sent = log_data_all
)
```

```r
results <- list()

for (name in names(datasets)) {

  df_current <- datasets[[name]]

  train_df <- df_current[train_index, ]
  test_df  <- df_current[-train_index, ]

  # Logistic
  model_logit <- glm(
    flags_recommend_frend ~ .,
    data = train_df,
```

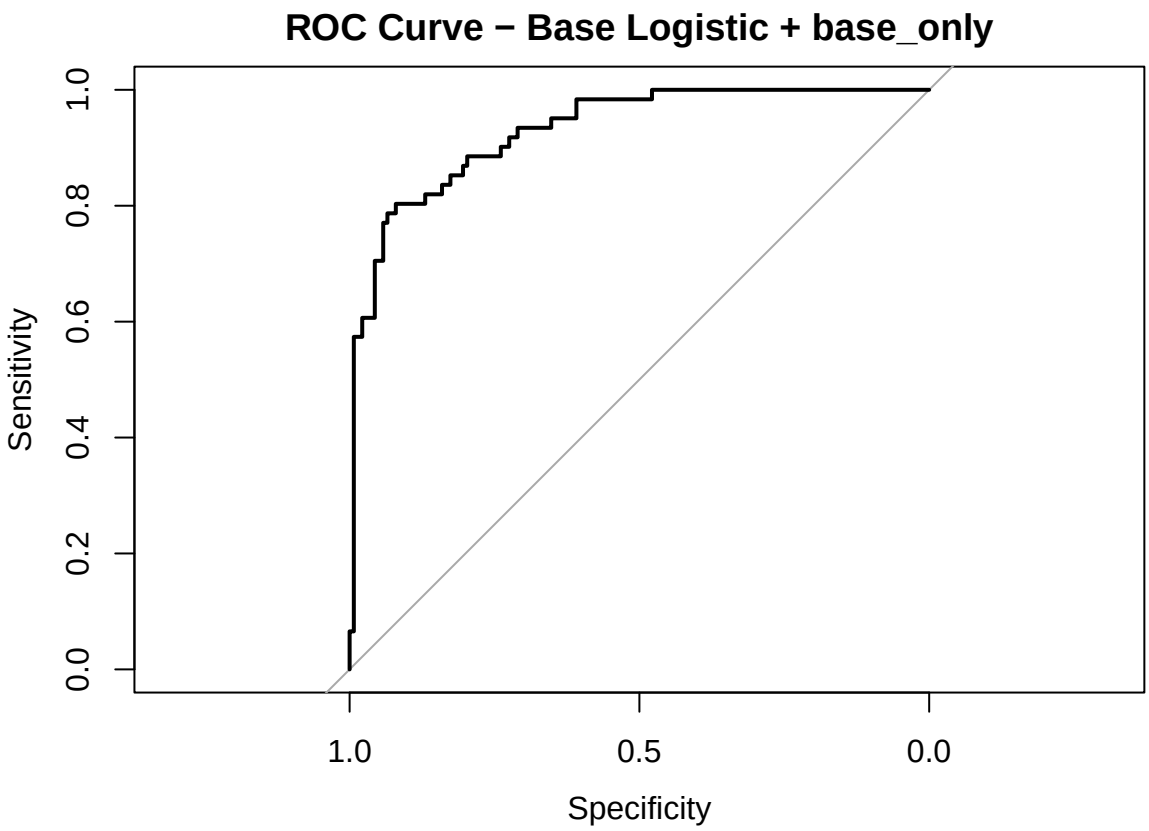
```r
  family = binomial
)

logit_prob <- predict(model_logit, newdata = test_df, type = "response")

logit_eval <- evaluate_model(
  prob = logit_prob,
  actual = test_df$flags_recommend_frend,
  model_name = paste("Base Logistic +", name)
)

# glmnet inputs
x_train <- model.matrix(flags_recommend_frend ~ ., data = train_df)[, -1]
x_test  <- model.matrix(flags_recommend_frend ~ ., data = test_df)[, -1]
y_train <- ifelse(train_df$flags_recommend_frend == "POSITIVE", 1, 0)

# LASSO
cv_lasso <- cv.glmnet(x_train, y_train, family = "binomial", alpha = 1)
lasso_prob <- as.numeric(predict(cv_lasso, newx = x_test, s = "lambda.min", type = "response"))

lasso_eval <- evaluate_model(
  prob = lasso_prob,
  actual = test_df$flags_recommend_frend,
  model_name = paste("LASSO +", name)
)

# Ridge
cv_ridge <- cv.glmnet(x_train, y_train, family = "binomial", alpha = 0)
ridge_prob <- as.numeric(predict(cv_ridge, newx = x_test, s = "lambda.min", type = "response"))

ridge_eval <- evaluate_model(
  prob = ridge_prob,
  actual = test_df$flags_recommend_frend,
  model_name = paste("Ridge +", name)
)

# Elastic Net
cv_enet <- cv.glmnet(x_train, y_train, family = "binomial", alpha = 0.5)
enet_prob <- as.numeric(predict(cv_enet, newx = x_test, s = "lambda.min", type = "response"))

enet_eval <- evaluate_model(
  prob = enet_prob,
  actual = test_df$flags_recommend_frend,
  model_name = paste("Elastic Net +", name)
)

# Store results
results[[name]] <- list(
  logit = logit_eval,
  lasso = lasso_eval,
  ridge = ridge_eval,
  enet  = enet_eval
)
```

```
}
```

```
## Best threshold for Base Logistic + base_only : 0.4
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      129       13
##   POSITIVE        9       48
##
##                Accuracy : 0.8894
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7351
##
##  Mcnemar's Test P-Value : 0.5224
##
##             Sensitivity : 0.9348
##             Specificity : 0.7869
##          Pos Pred Value : 0.9085
##          Neg Pred Value : 0.8421
##              Prevalence : 0.6935
##          Detection Rate : 0.6482
##    Detection Prevalence : 0.7136
##       Balanced Accuracy : 0.8608
##
##        'Positive' Class : NEGATIVE
##
```

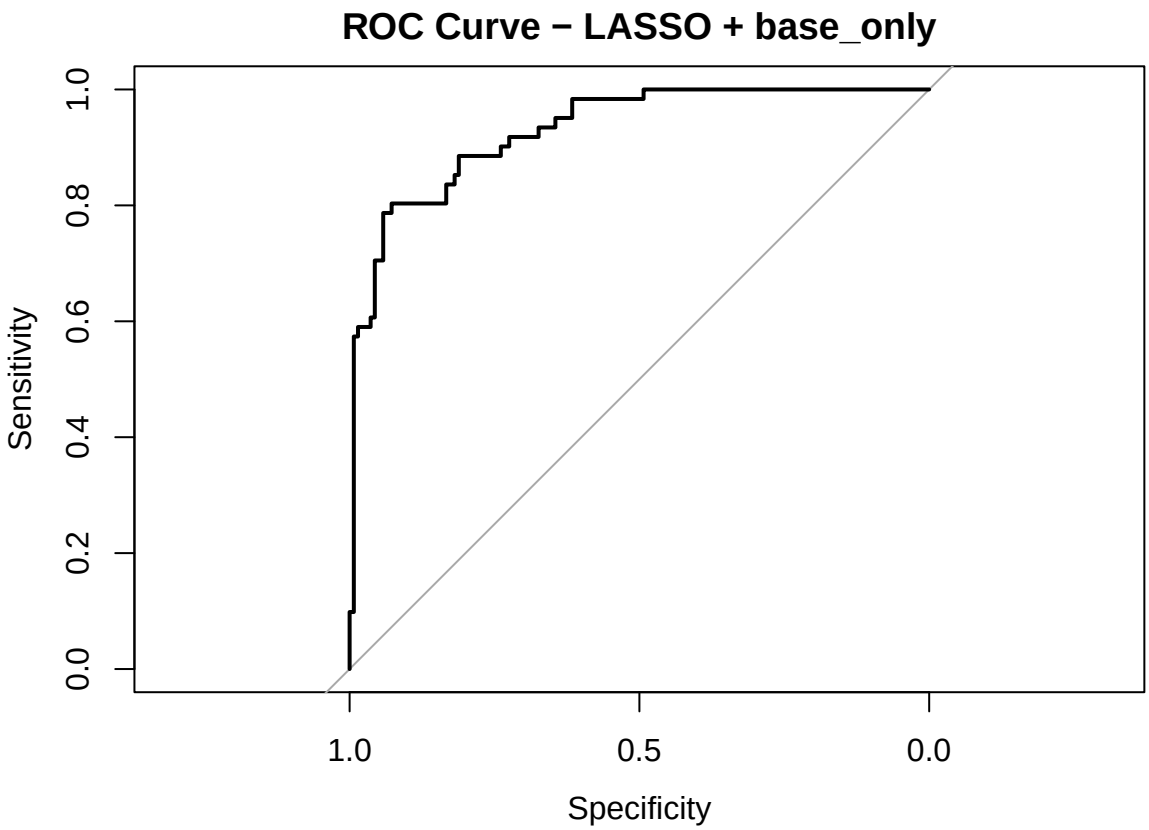
```
## Setting levels: control = NEGATIVE, case = POSITIVE
```

```
## Setting direction: controls < cases
```




```
## Base Logistic + base_only AUC: 0.9303873
## Best threshold for LASSO + base_only : 0.45
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      130       13
##   POSITIVE        8       48
##
##                Accuracy : 0.8945
##                  95% CI : (0.8432, 0.9335)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.388e-11
##
##                   Kappa : 0.746
##
##  Mcnemar's Test P-Value : 0.3827
##
##             Sensitivity : 0.9420
##             Specificity : 0.7869
##          Pos Pred Value : 0.9091
##          Neg Pred Value : 0.8571
##              Prevalence : 0.6935
##          Detection Rate : 0.6533
##    Detection Prevalence : 0.7186
##       Balanced Accuracy : 0.8645
```

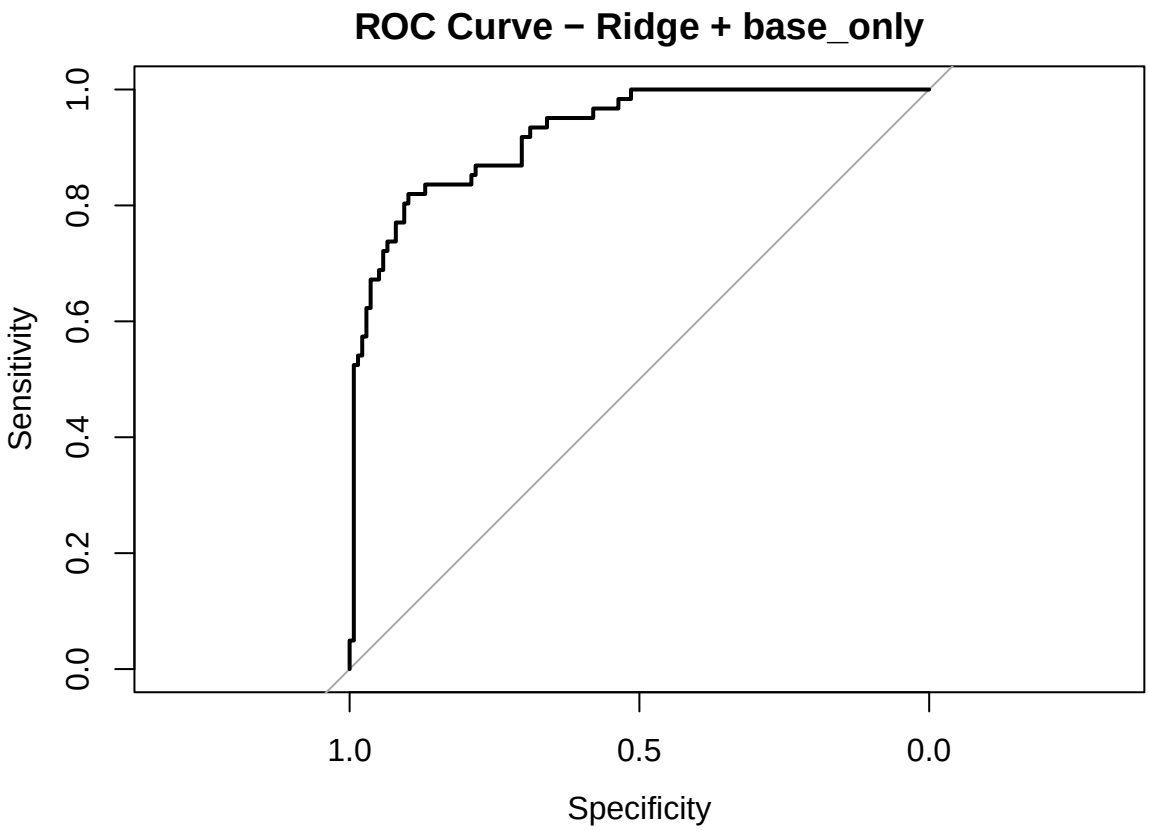
```
##
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## LASSO + base_only AUC: 0.9300309
## Best threshold for Ridge + base_only : 0.35
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      124       11
##   POSITIVE       14       50
##
##                Accuracy : 0.8744
##                  95% CI : (0.8202, 0.917)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.833e-09
##
##                   Kappa : 0.7085
##
##  Mcnemar's Test P-Value : 0.6892
##
```

```
##             Sensitivity : 0.8986
##             Specificity : 0.8197
##          Pos Pred Value : 0.9185
##          Neg Pred Value : 0.7812
##              Prevalence : 0.6935
##          Detection Rate : 0.6231
##    Detection Prevalence : 0.6784
##       Balanced Accuracy : 0.8591
##
##        'Positive' Class : NEGATIVE
##
```

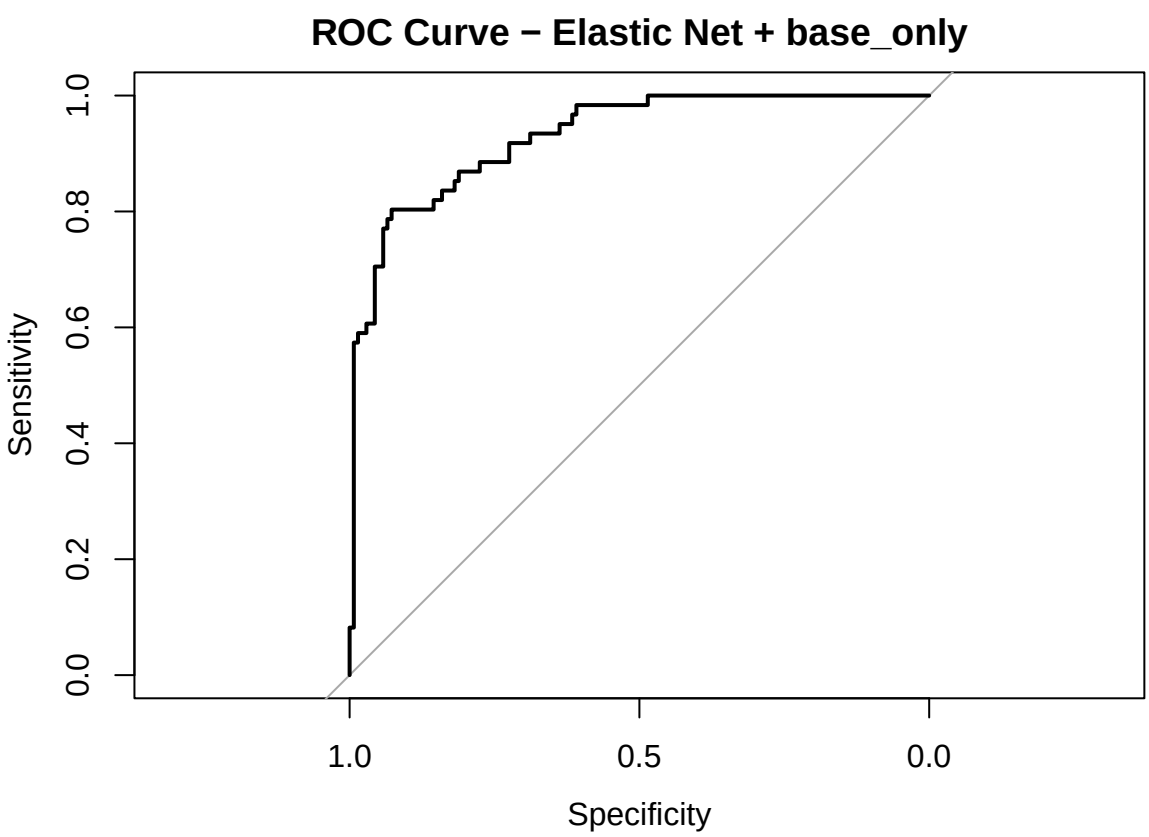
```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Ridge + base_only AUC: 0.9243288
## Best threshold for Elastic Net + base_only : 0.35
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      128       12
##   POSITIVE       10       49
##
##                Accuracy : 0.8894
```

```
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7376
##
##  Mcnemar's Test P-Value : 0.8312
##
##             Sensitivity : 0.9275
##             Specificity : 0.8033
##          Pos Pred Value : 0.9143
##          Neg Pred Value : 0.8305
##              Prevalence : 0.6935
##          Detection Rate : 0.6432
##    Detection Prevalence : 0.7035
##       Balanced Accuracy : 0.8654
##
##        'Positive' Class : NEGATIVE
##
```

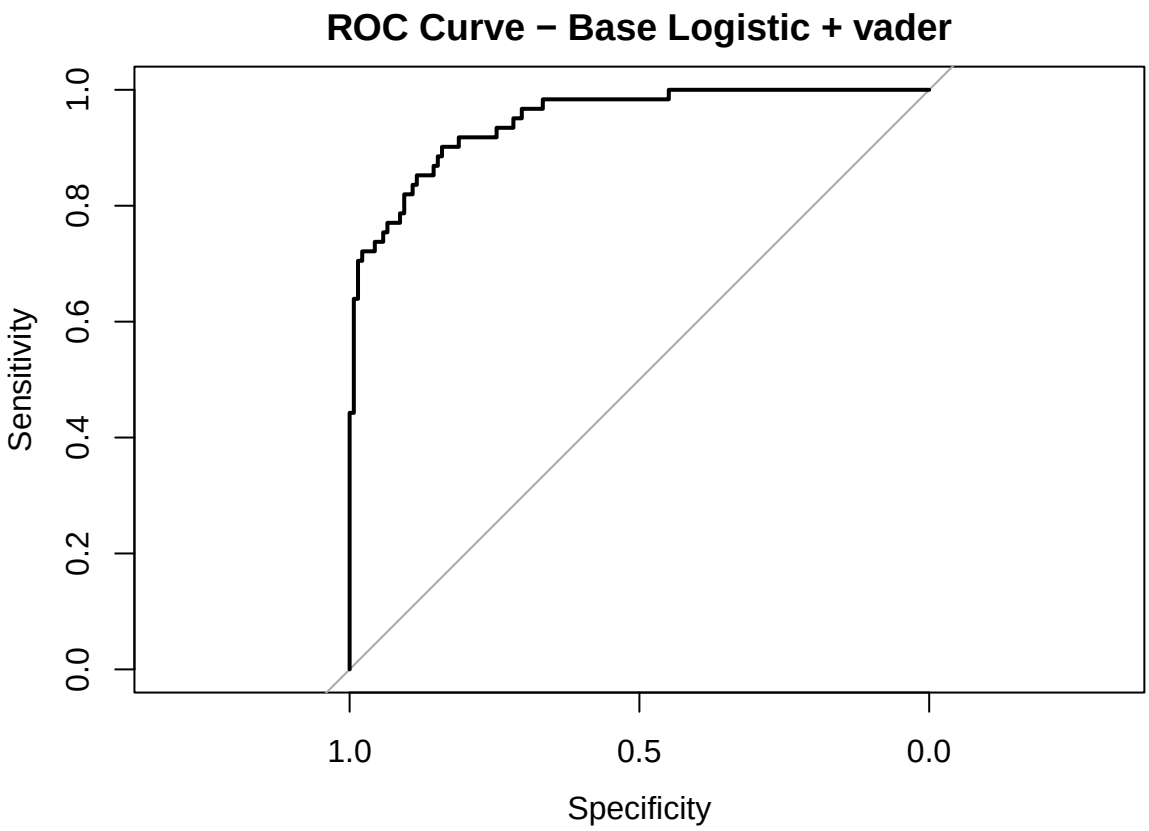
```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Elastic Net + base_only AUC: 0.9294369
## Best threshold for Base Logistic + vader : 0.6
```

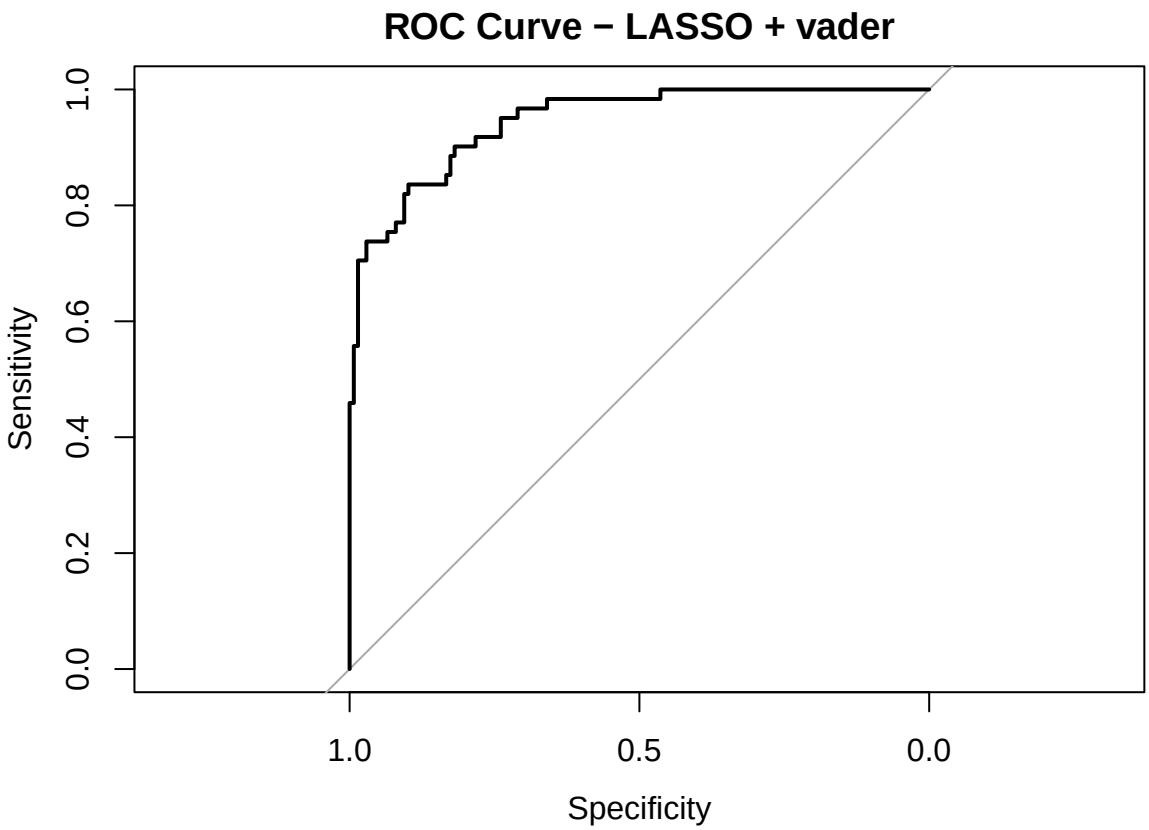
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      134       17
##   POSITIVE        4       44
##
##                Accuracy : 0.8945
##                  95% CI : (0.8432, 0.9335)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.388e-11
##
##                   Kappa : 0.7361
##
##  Mcnemar's Test P-Value : 0.008829
##
##             Sensitivity : 0.9710
##             Specificity : 0.7213
##          Pos Pred Value : 0.8874
##          Neg Pred Value : 0.9167
##              Prevalence : 0.6935
##          Detection Rate : 0.6734
##    Detection Prevalence : 0.7588
##       Balanced Accuracy : 0.8462
##
##        'Positive' Class : NEGATIVE
##
```

```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```



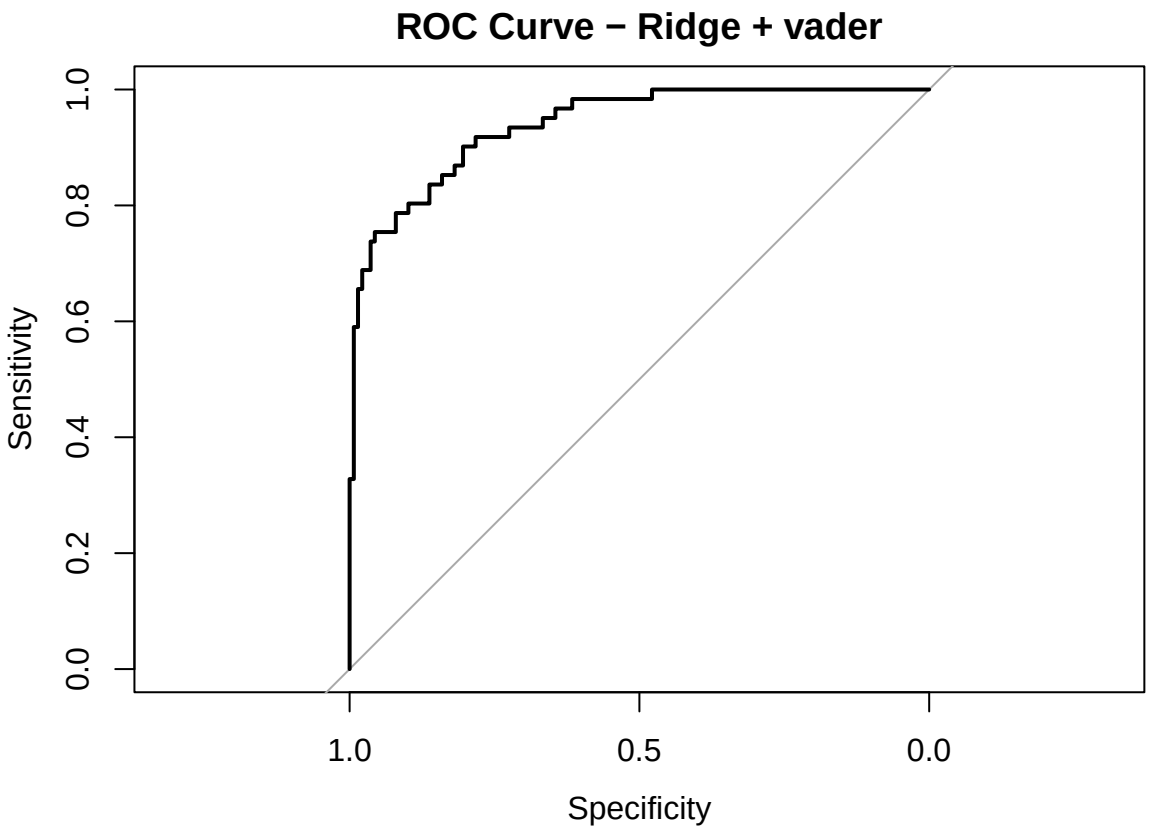

```
## Base Logistic + vader AUC: 0.9476123
## Best threshold for LASSO + vader : 0.65
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      136       18
##   POSITIVE        2       43
##
##                Accuracy : 0.8995
##                  95% CI : (0.8491, 0.9375)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 3.617e-12
##
##                   Kappa : 0.7449
##
##  Mcnemar's Test P-Value : 0.0007962
##
##             Sensitivity : 0.9855
##             Specificity : 0.7049
##          Pos Pred Value : 0.8831
##          Neg Pred Value : 0.9556
##              Prevalence : 0.6935
##          Detection Rate : 0.6834
##    Detection Prevalence : 0.7739
##       Balanced Accuracy : 0.8452
```

```
##
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## LASSO + vader AUC: 0.94488
## Best threshold for Ridge + vader : 0.45
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      131       15
##   POSITIVE        7       46
##
##                Accuracy : 0.8894
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7301
##
##  Mcnemar's Test P-Value : 0.1356
##
```

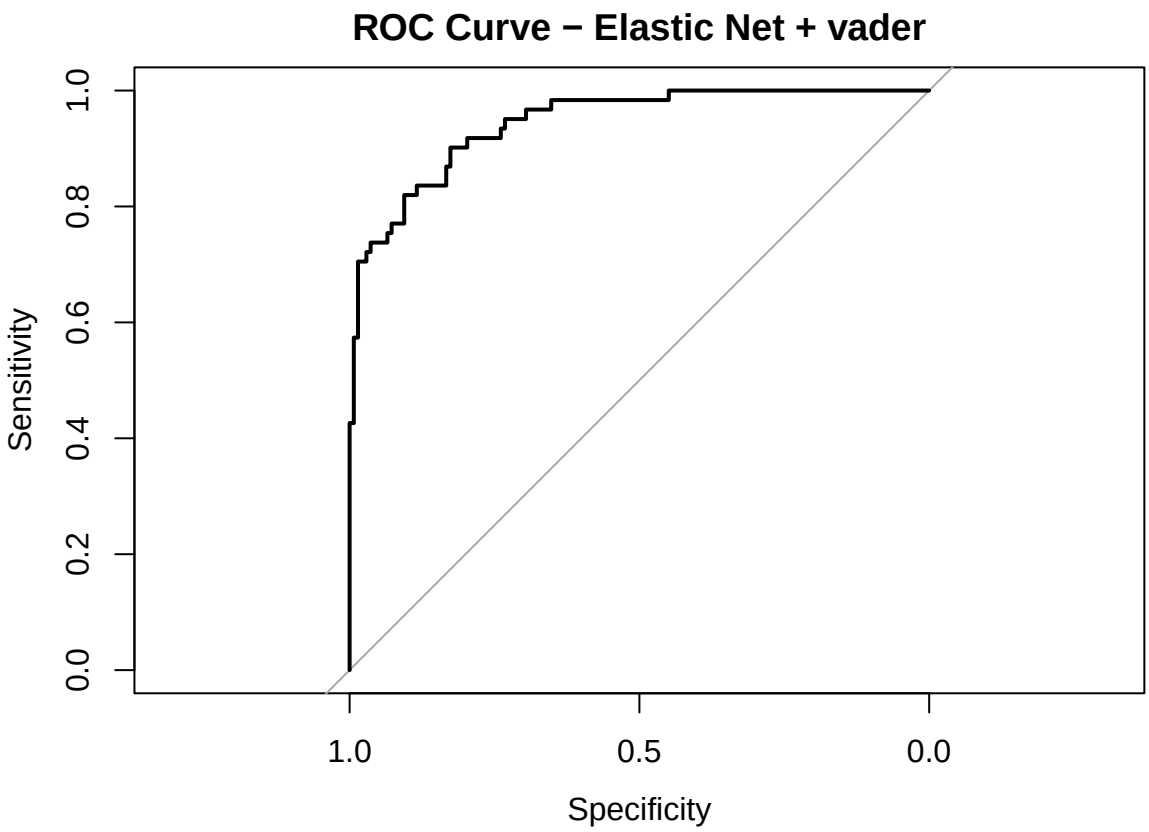
```
##             Sensitivity : 0.9493
##             Specificity : 0.7541
##          Pos Pred Value : 0.8973
##          Neg Pred Value : 0.8679
##              Prevalence : 0.6935
##          Detection Rate : 0.6583
##    Detection Prevalence : 0.7337
##       Balanced Accuracy : 0.8517
##
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Ridge + vader AUC: 0.9389404
## Best threshold for Elastic Net + vader : 0.65
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      136       18
##   POSITIVE        2       43
##
##                Accuracy : 0.8995
```

```
##                  95% CI : (0.8491, 0.9375)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 3.617e-12
##
##                   Kappa : 0.7449
##
##  Mcnemar's Test P-Value : 0.0007962
##
##             Sensitivity : 0.9855
##             Specificity : 0.7049
##          Pos Pred Value : 0.8831
##          Neg Pred Value : 0.9556
##              Prevalence : 0.6935
##          Detection Rate : 0.6834
##    Detection Prevalence : 0.7739
##       Balanced Accuracy : 0.8452
##
##        'Positive' Class : NEGATIVE
##
```

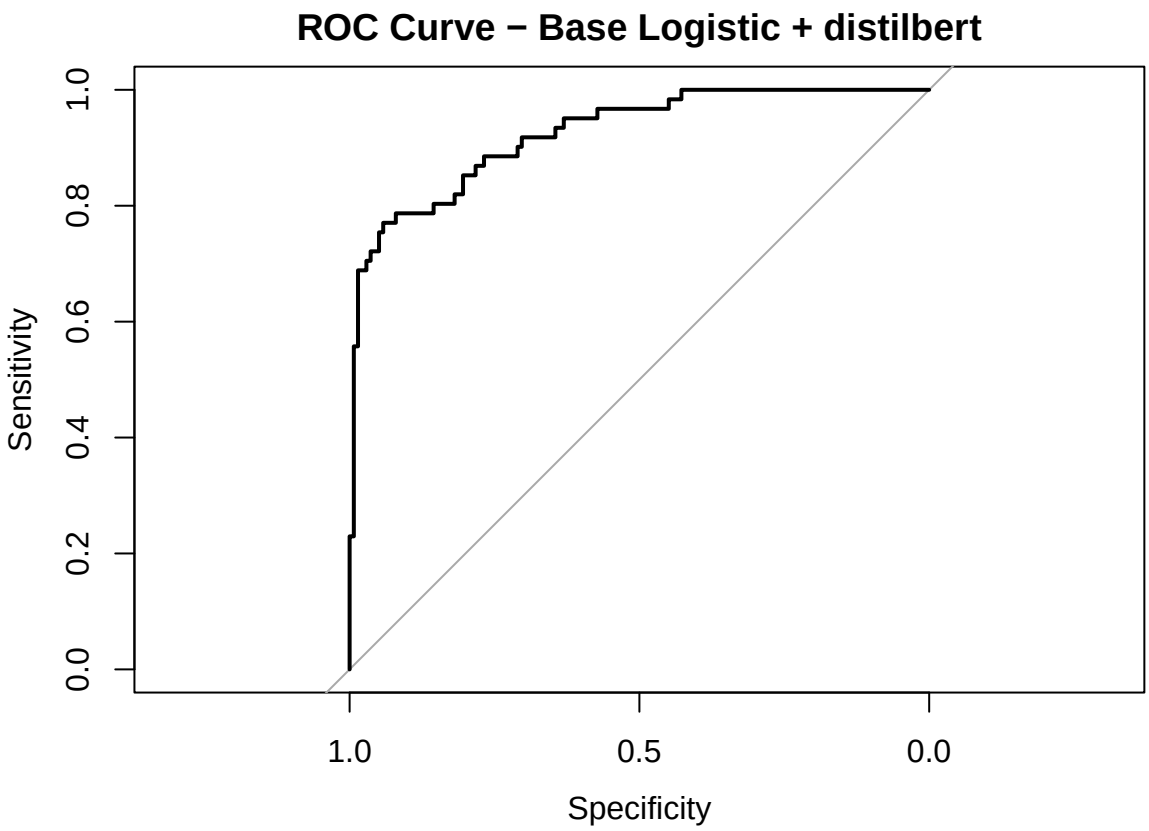
```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Elastic Net + vader AUC: 0.9442861
## Best threshold for Base Logistic + distilbert : 0.5
```

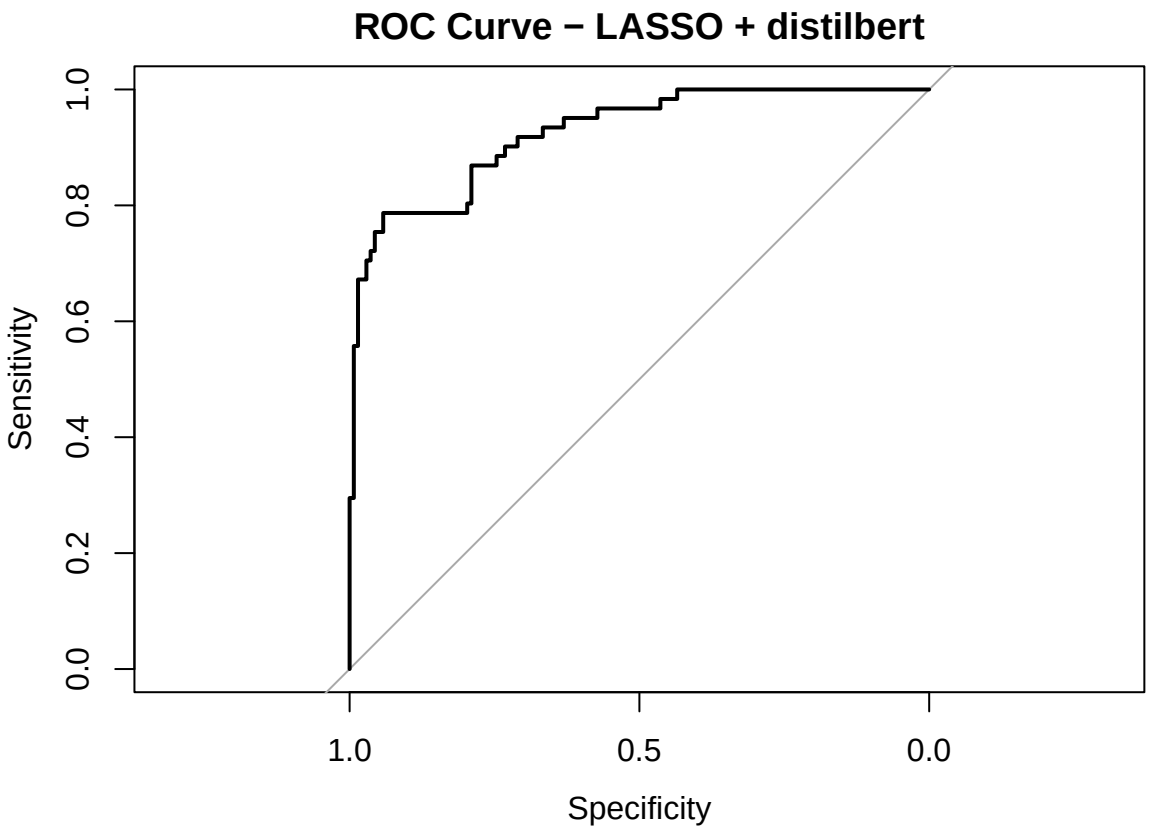
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      131       15
##   POSITIVE        7       46
##
##                Accuracy : 0.8894
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7301
##
##  Mcnemar's Test P-Value : 0.1356
##
##             Sensitivity : 0.9493
##             Specificity : 0.7541
##          Pos Pred Value : 0.8973
##          Neg Pred Value : 0.8679
##              Prevalence : 0.6935
##          Detection Rate : 0.6583
##    Detection Prevalence : 0.7337
##       Balanced Accuracy : 0.8517
##
##        'Positive' Class : NEGATIVE
##
```

```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```



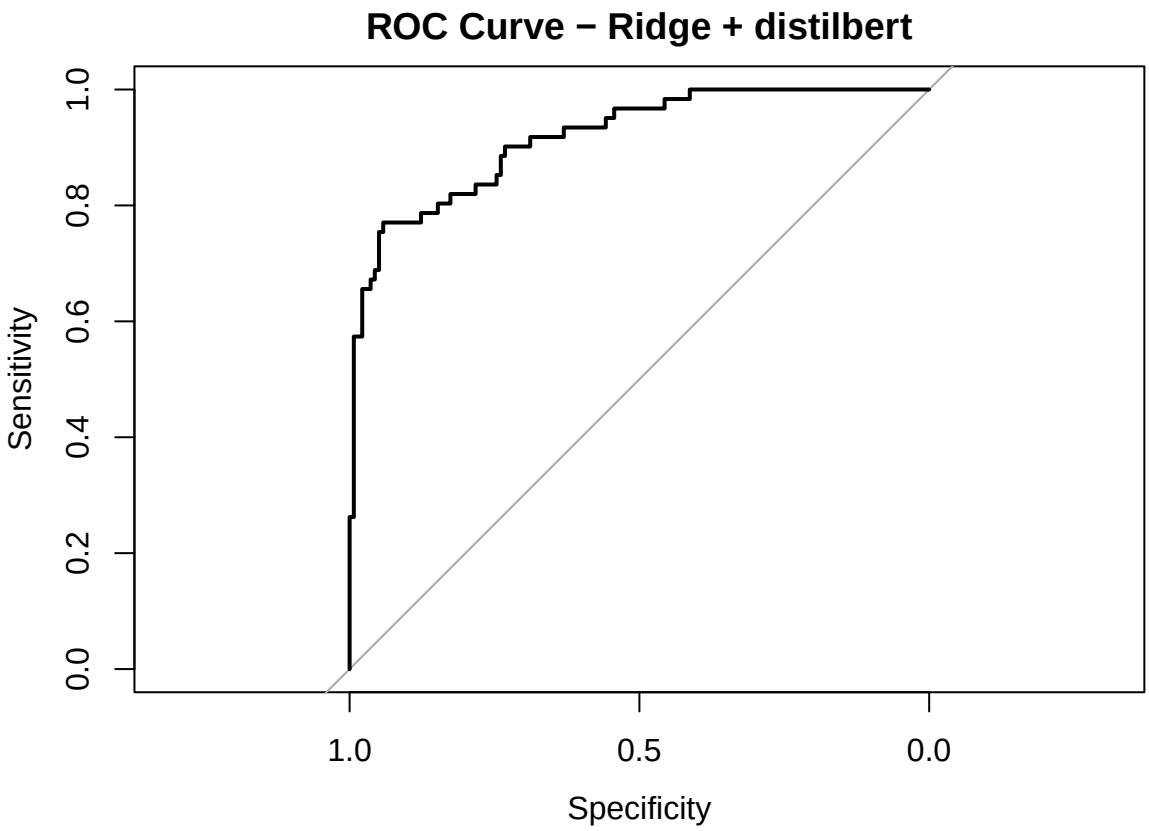

```
## Base Logistic + distilbert AUC: 0.9246852
## Best threshold for LASSO + distilbert : 0.4
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      130       14
##   POSITIVE        8       47
##
##                Accuracy : 0.8894
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7326
##
##  Mcnemar's Test P-Value : 0.2864
##
##             Sensitivity : 0.9420
##             Specificity : 0.7705
##          Pos Pred Value : 0.9028
##          Neg Pred Value : 0.8545
##              Prevalence : 0.6935
##          Detection Rate : 0.6533
##    Detection Prevalence : 0.7236
##       Balanced Accuracy : 0.8563
```

```
##
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## LASSO + distilbert AUC: 0.9245664
## Best threshold for Ridge + distilbert : 0.5
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      131       15
##   POSITIVE        7       46
##
##                Accuracy : 0.8894
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7301
##
##  Mcnemar's Test P-Value : 0.1356
##
```

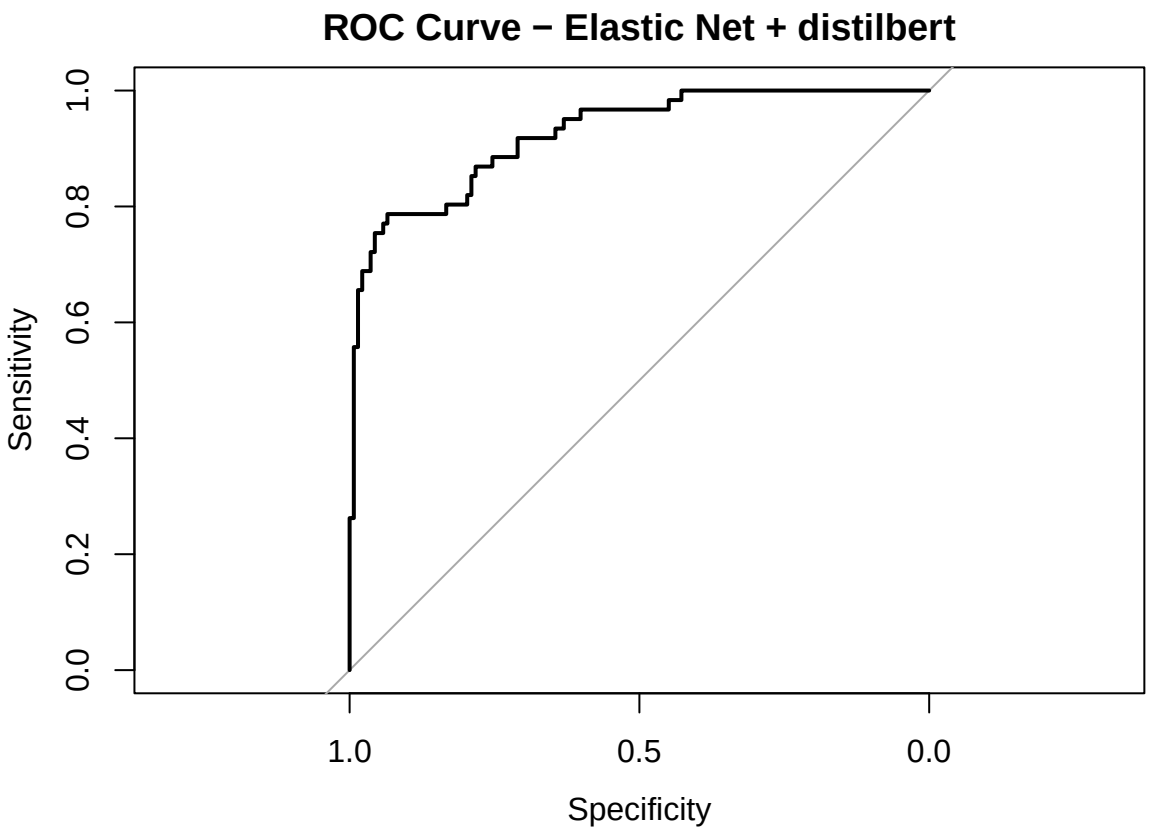
```
##             Sensitivity : 0.9493
##             Specificity : 0.7541
##          Pos Pred Value : 0.8973
##          Neg Pred Value : 0.8679
##              Prevalence : 0.6935
##          Detection Rate : 0.6583
##    Detection Prevalence : 0.7337
##       Balanced Accuracy : 0.8517
##
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Ridge + distilbert AUC: 0.917914
## Best threshold for Elastic Net + distilbert : 0.4
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      130       14
##   POSITIVE        8       47
##
##                Accuracy : 0.8894
```

```
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7326
##
##  Mcnemar's Test P-Value : 0.2864
##
##             Sensitivity : 0.9420
##             Specificity : 0.7705
##          Pos Pred Value : 0.9028
##          Neg Pred Value : 0.8545
##              Prevalence : 0.6935
##          Detection Rate : 0.6533
##    Detection Prevalence : 0.7236
##       Balanced Accuracy : 0.8563
##
##        'Positive' Class : NEGATIVE
##
```

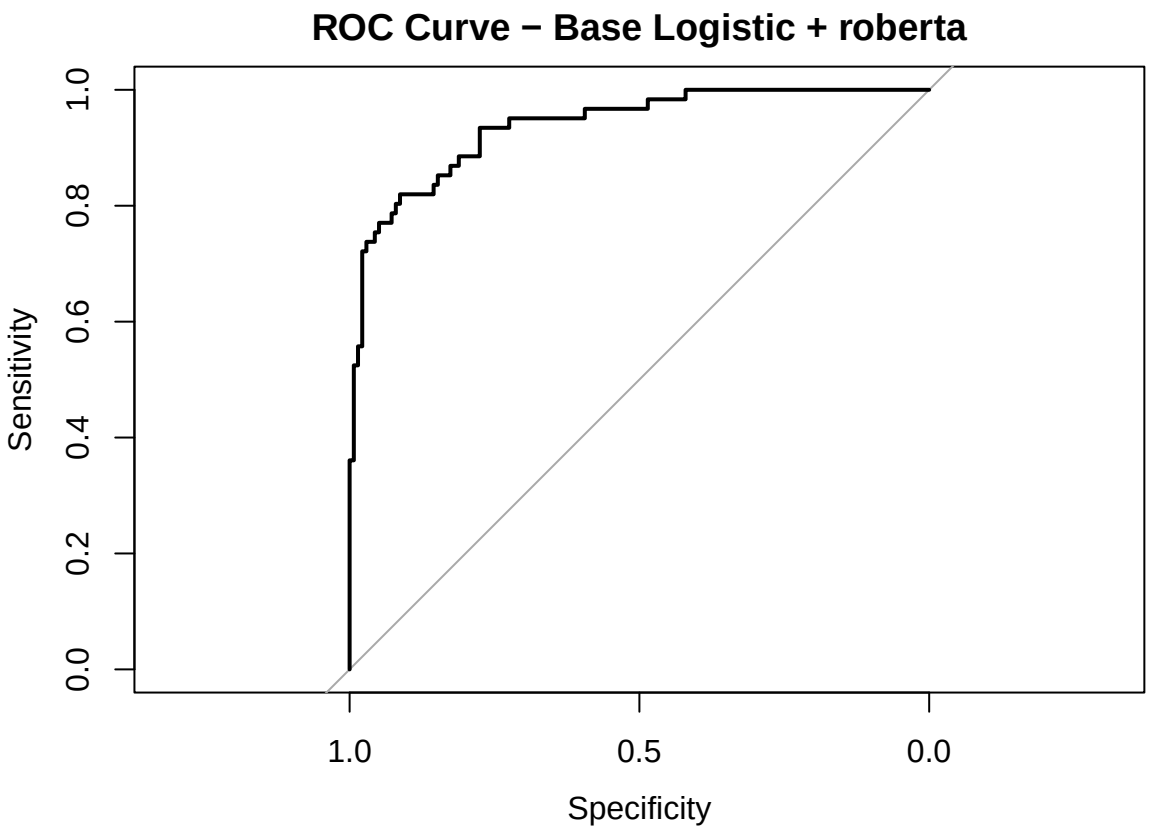
```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Elastic Net + distilbert AUC: 0.92421
## Best threshold for Base Logistic + roberta : 0.45
```

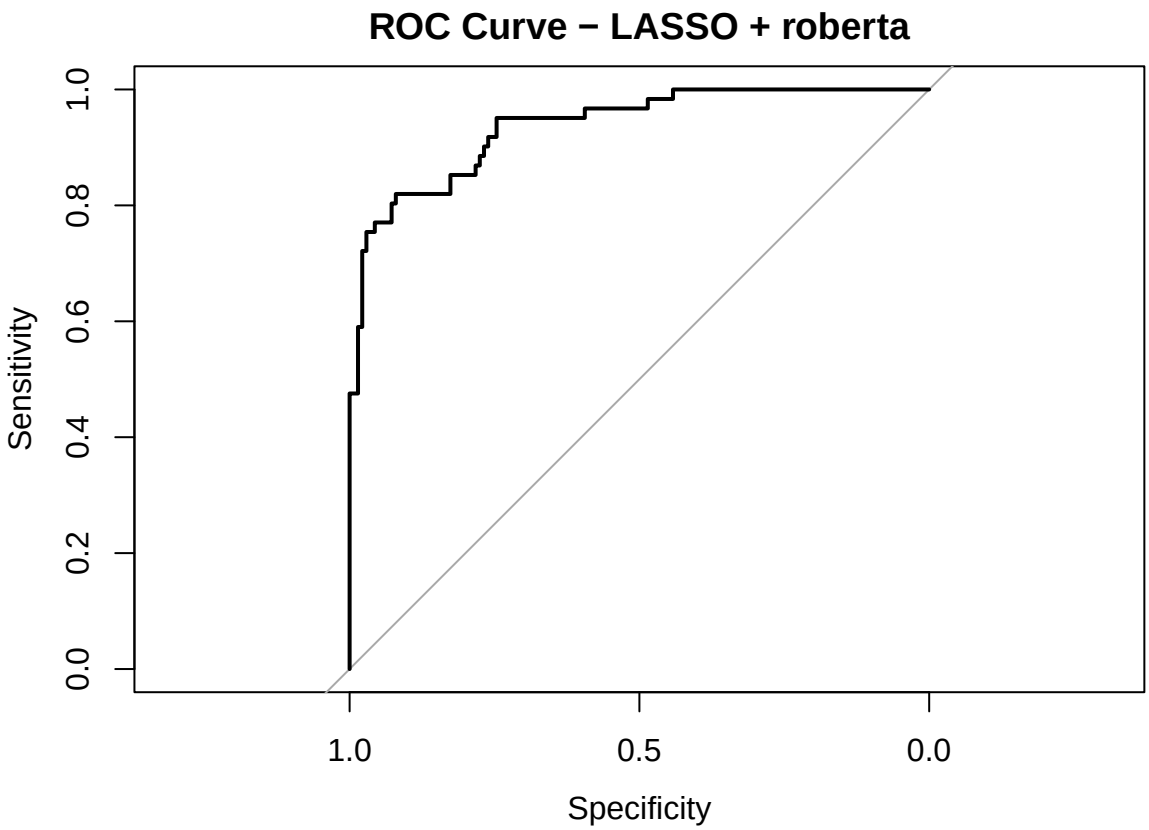
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      131       14
##   POSITIVE        7       47
##
##                Accuracy : 0.8945
##                  95% CI : (0.8432, 0.9335)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.388e-11
##
##                   Kappa : 0.7436
##
##  Mcnemar's Test P-Value : 0.1904
##
##             Sensitivity : 0.9493
##             Specificity : 0.7705
##          Pos Pred Value : 0.9034
##          Neg Pred Value : 0.8704
##              Prevalence : 0.6935
##          Detection Rate : 0.6583
##    Detection Prevalence : 0.7286
##       Balanced Accuracy : 0.8599
##
##        'Positive' Class : NEGATIVE
##
```

```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Base Logistic + roberta AUC: 0.9378712
## Best threshold for LASSO + roberta : 0.5
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      132       14
##   POSITIVE        6       47
##
##                Accuracy : 0.8995
##                  95% CI : (0.8491, 0.9375)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 3.617e-12
##
##                   Kappa : 0.7546
##
##  Mcnemar's Test P-Value : 0.1175
##
##             Sensitivity : 0.9565
##             Specificity : 0.7705
##          Pos Pred Value : 0.9041
##          Neg Pred Value : 0.8868
##              Prevalence : 0.6935
##          Detection Rate : 0.6633
##    Detection Prevalence : 0.7337
##       Balanced Accuracy : 0.8635
```

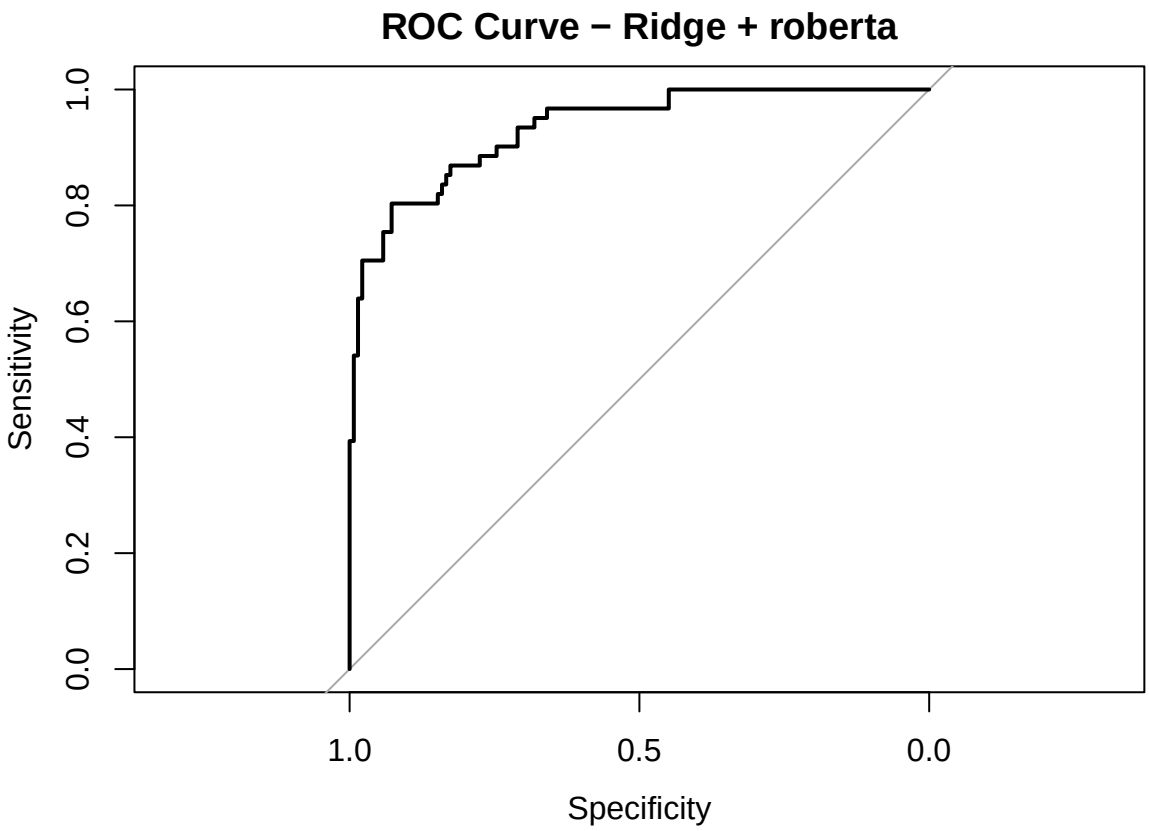
```
##
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## LASSO + roberta AUC: 0.9369209
## Best threshold for Ridge + roberta : 0.35
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      126       12
##   POSITIVE       12       49
##
##                Accuracy : 0.8794
##                  95% CI : (0.8259, 0.9212)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.802e-10
##
##                   Kappa : 0.7163
##
##  Mcnemar's Test P-Value : 1
##
```

```
##             Sensitivity : 0.9130
##             Specificity : 0.8033
##          Pos Pred Value : 0.9130
##          Neg Pred Value : 0.8033
##              Prevalence : 0.6935
##          Detection Rate : 0.6332
##    Detection Prevalence : 0.6935
##       Balanced Accuracy : 0.8582
##
##        'Positive' Class : NEGATIVE
##
```

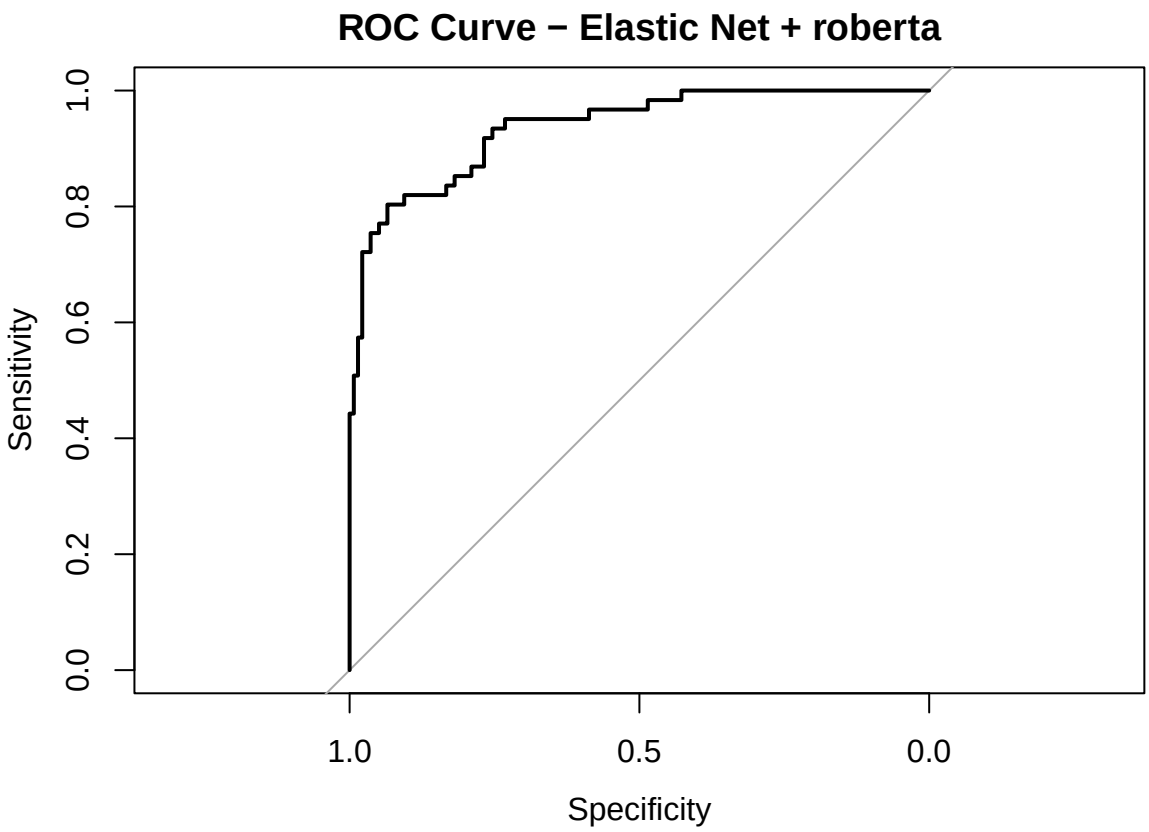
```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Ridge + roberta AUC: 0.9327631
## Best threshold for Elastic Net + roberta : 0.35
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      128       12
##   POSITIVE       10       49
##
##                Accuracy : 0.8894
```

```
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7376
##
##  Mcnemar's Test P-Value : 0.8312
##
##             Sensitivity : 0.9275
##             Specificity : 0.8033
##          Pos Pred Value : 0.9143
##          Neg Pred Value : 0.8305
##              Prevalence : 0.6935
##          Detection Rate : 0.6432
##    Detection Prevalence : 0.7035
##       Balanced Accuracy : 0.8654
##
##        'Positive' Class : NEGATIVE
##
```

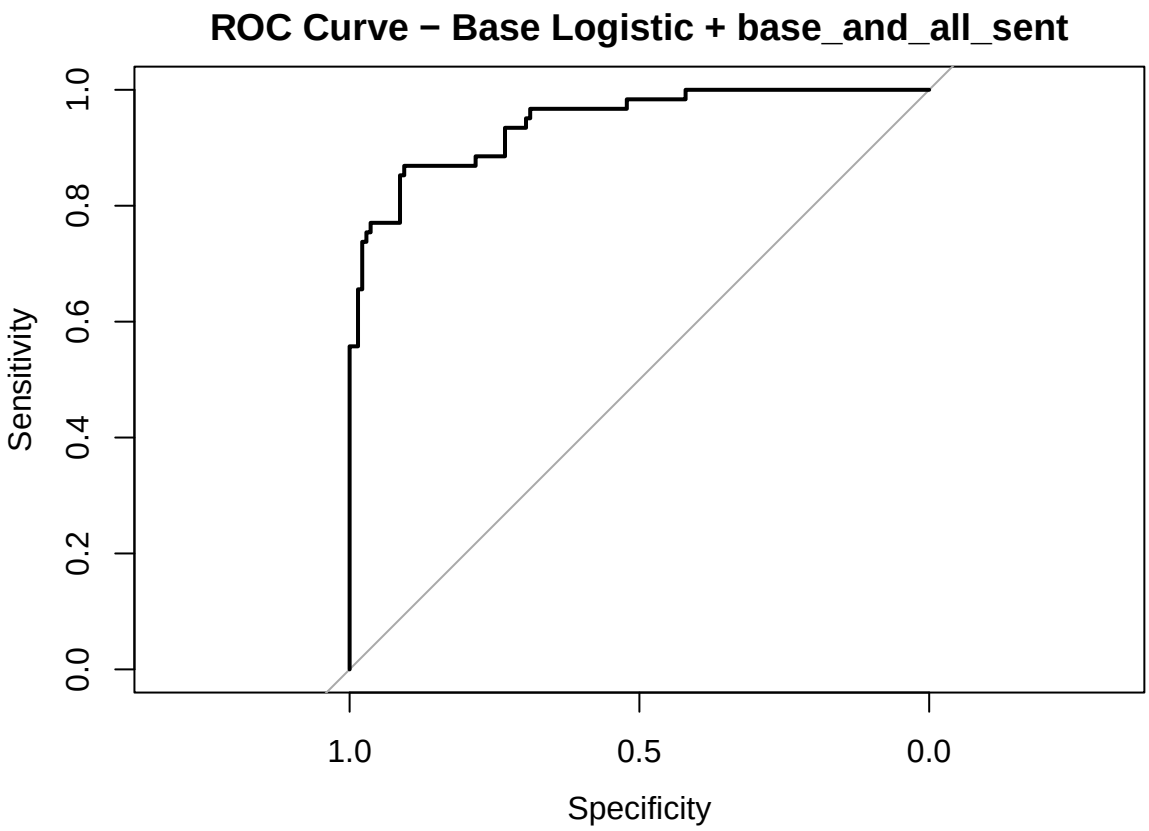
```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Elastic Net + roberta AUC: 0.9360893
## Best threshold for Base Logistic + base_and_all_sent : 0.45
```

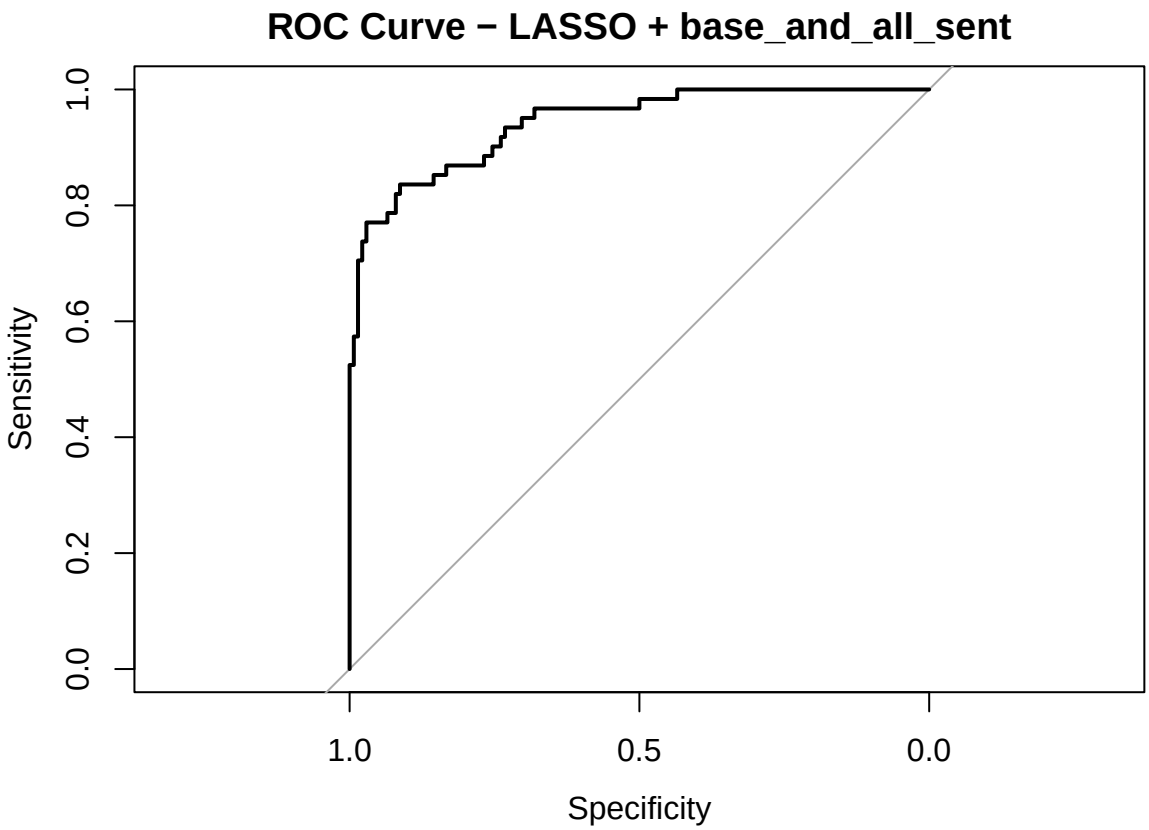
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      133       14
##   POSITIVE        5       47
##
##                Accuracy : 0.9045
##                  95% CI : (0.8549, 0.9415)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 8.932e-13
##
##                   Kappa : 0.7658
##
##  Mcnemar's Test P-Value : 0.06646
##
##             Sensitivity : 0.9638
##             Specificity : 0.7705
##          Pos Pred Value : 0.9048
##          Neg Pred Value : 0.9038
##              Prevalence : 0.6935
##          Detection Rate : 0.6683
##    Detection Prevalence : 0.7387
##       Balanced Accuracy : 0.8671
##
##        'Positive' Class : NEGATIVE
##
```

```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Base Logistic + base_and_all_sent AUC: 0.9428605
## Best threshold for LASSO + base_and_all_sent : 0.5
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      134       14
##   POSITIVE        4       47
##
##                Accuracy : 0.9095
##                  95% CI : (0.8608, 0.9455)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 2.085e-13
##
##                   Kappa : 0.777
##
##  Mcnemar's Test P-Value : 0.03389
##
##             Sensitivity : 0.9710
##             Specificity : 0.7705
##          Pos Pred Value : 0.9054
##          Neg Pred Value : 0.9216
##              Prevalence : 0.6935
##          Detection Rate : 0.6734
##    Detection Prevalence : 0.7437
##       Balanced Accuracy : 0.8708
```

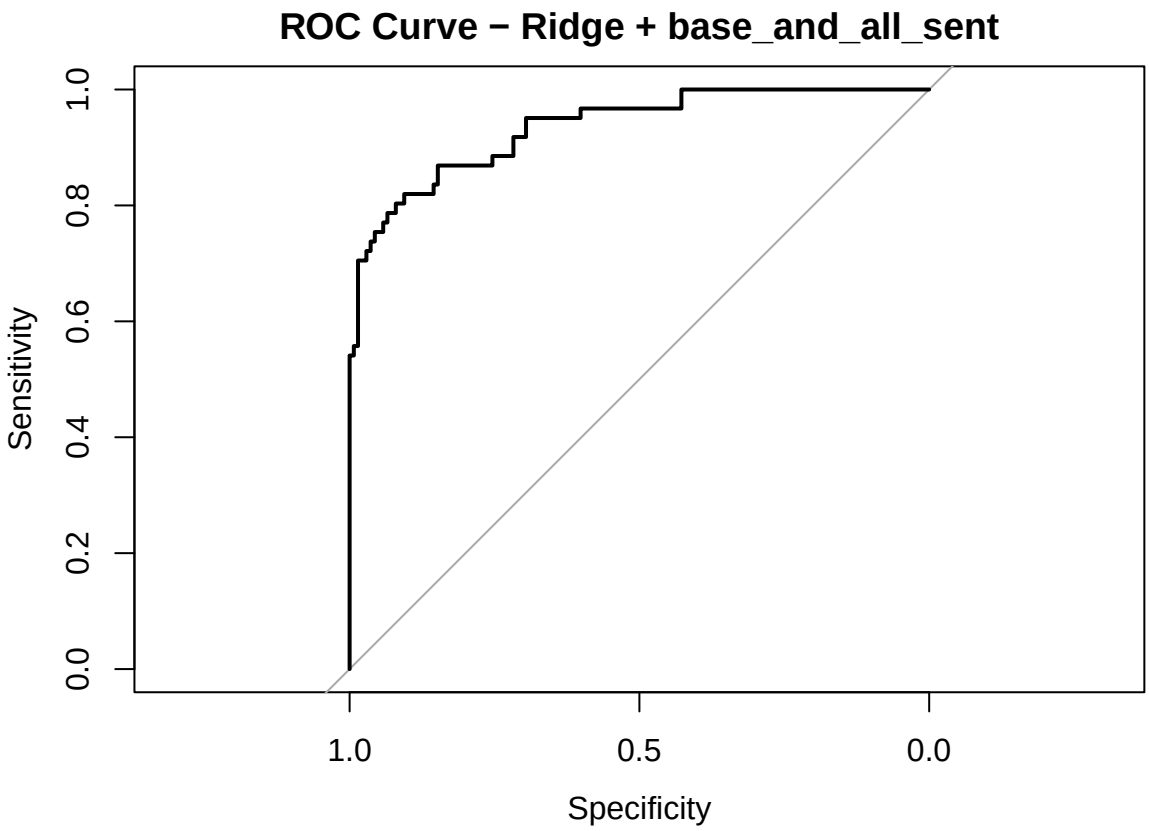
```
##
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## LASSO + base_and_all_sent AUC: 0.9417914
## Best threshold for Ridge + base_and_all_sent : 0.45
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      132       15
##   POSITIVE        6       46
##
##                Accuracy : 0.8945
##                  95% CI : (0.8432, 0.9335)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.388e-11
##
##                   Kappa : 0.7411
##
##  Mcnemar's Test P-Value : 0.08086
##
```

```
##             Sensitivity : 0.9565
##             Specificity : 0.7541
##          Pos Pred Value : 0.8980
##          Neg Pred Value : 0.8846
##              Prevalence : 0.6935
##          Detection Rate : 0.6633
##    Detection Prevalence : 0.7387
##       Balanced Accuracy : 0.8553
##
##        'Positive' Class : NEGATIVE
##
```

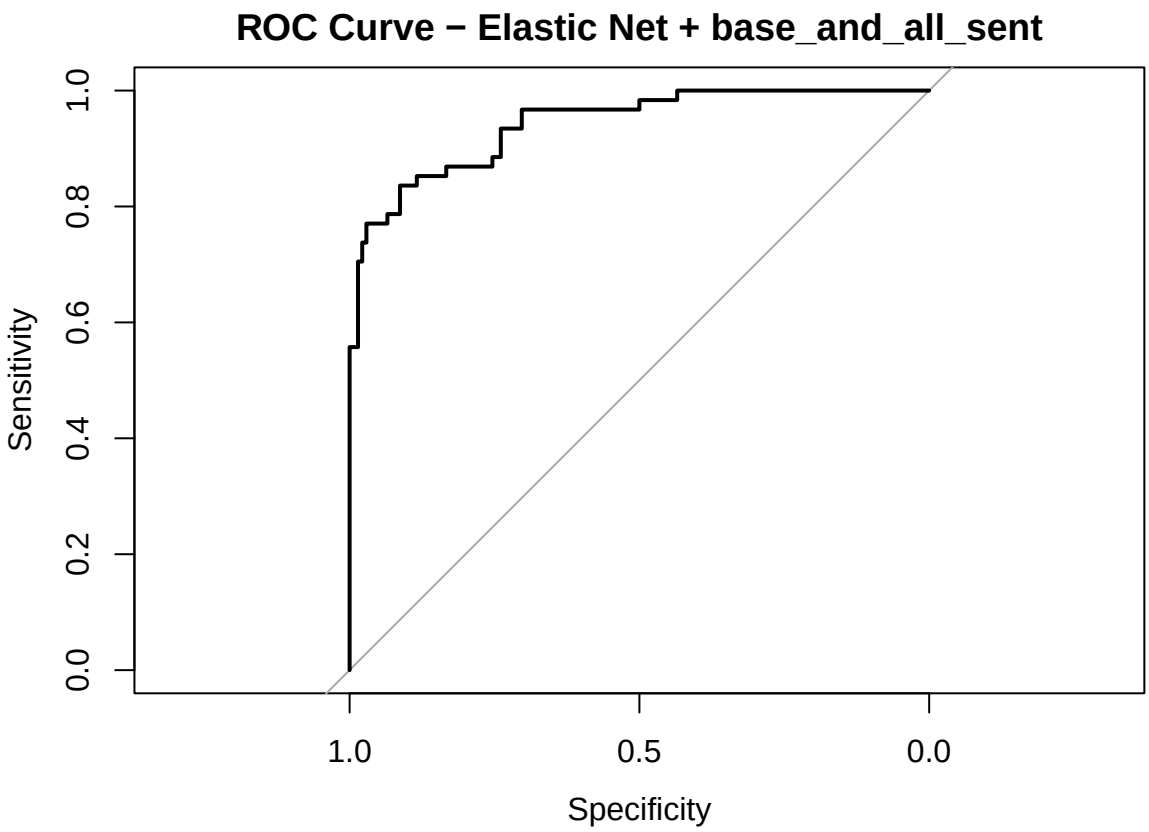
```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Ridge + base_and_all_sent AUC: 0.935139
## Best threshold for Elastic Net + base_and_all_sent : 0.5
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      133       14
##   POSITIVE        5       47
##
##                Accuracy : 0.9045
```

```
##                  95% CI : (0.8549, 0.9415)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 8.932e-13
##
##                   Kappa : 0.7658
##
##  Mcnemar's Test P-Value : 0.06646
##
##             Sensitivity : 0.9638
##             Specificity : 0.7705
##          Pos Pred Value : 0.9048
##          Neg Pred Value : 0.9038
##              Prevalence : 0.6935
##          Detection Rate : 0.6683
##    Detection Prevalence : 0.7387
##       Balanced Accuracy : 0.8671
##
##        'Positive' Class : NEGATIVE
##
```

```
## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## Elastic Net + base_and_all_sent AUC: 0.9421478
```

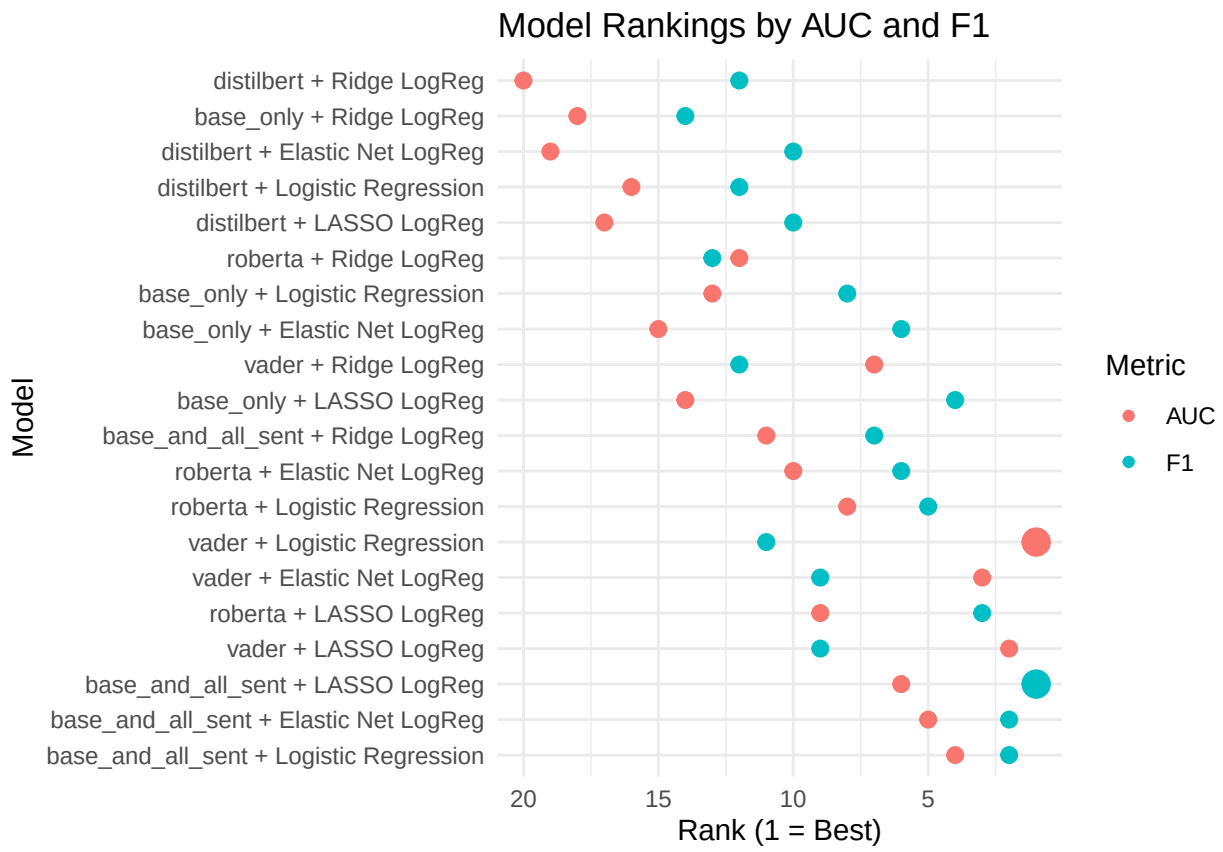
```r
model_comparison <- do.call(rbind, lapply(names(results), function(name) {

  res <- results[[name]]

  data.frame(
    Features = name,
    Model = c("Logistic Regression", "LASSO LogReg", "Ridge LogReg", "Elastic Net LogReg"),

    AUC = c(
      as.numeric(res$logit$auc),
      as.numeric(res$lasso$auc),
      as.numeric(res$ridge$auc),
      as.numeric(res$enet$auc)
    ),

    F1 = c(
      max(res$logit$f1_results$F1),
      max(res$lasso$f1_results$F1),
      max(res$ridge$f1_results$F1),
      max(res$enet$f1_results$F1)
    ),

    Best_Threshold = c(
      res$logit$best_threshold,
      res$lasso$best_threshold,
      res$ridge$best_threshold,
      res$enet$best_threshold
    ),

    Accuracy = c(
      as.numeric(res$logit$confusion_matrix$overall["Accuracy"]),
      as.numeric(res$lasso$confusion_matrix$overall["Accuracy"]),
      as.numeric(res$ridge$confusion_matrix$overall["Accuracy"]),
      as.numeric(res$enet$confusion_matrix$overall["Accuracy"])
    )
  )
}))

model_comparison %>% arrange(desc(AUC))
```

```
##             Features               Model       AUC        F1 Best_Threshold
## 1              vader Logistic Regression 0.9476123 0.8073394           0.60
## 2              vader        LASSO LogReg 0.9448800 0.8113208           0.65
## 3              vader  Elastic Net LogReg 0.9442861 0.8113208           0.65
## 4  base_and_all_sent Logistic Regression 0.9428605 0.8318584           0.45
## 5  base_and_all_sent  Elastic Net LogReg 0.9421478 0.8318584           0.50
## 6  base_and_all_sent        LASSO LogReg 0.9417914 0.8392857           0.50
## 7              vader        Ridge LogReg 0.9389404 0.8070175           0.45
## 8            roberta Logistic Regression 0.9378712 0.8173913           0.45
## 9            roberta        LASSO LogReg 0.9369209 0.8245614           0.50
## 10           roberta  Elastic Net LogReg 0.9360893 0.8166667           0.35
## 11 base_and_all_sent        Ridge LogReg 0.9351390 0.8141593           0.45
## 12           roberta        Ridge LogReg 0.9327631 0.8032787           0.35
```

```
## 13         base_only Logistic Regression 0.9303873 0.8135593           0.40
## 14         base_only        LASSO LogReg 0.9300309 0.8205128           0.45
## 15         base_only  Elastic Net LogReg 0.9294369 0.8166667           0.35
## 16        distilbert Logistic Regression 0.9246852 0.8070175           0.50
## 17        distilbert        LASSO LogReg 0.9245664 0.8103448           0.40
## 18         base_only        Ridge LogReg 0.9243288 0.8000000           0.35
## 19        distilbert  Elastic Net LogReg 0.9242100 0.8103448           0.40
## 20        distilbert        Ridge LogReg 0.9179140 0.8070175           0.50
##     Accuracy
## 1  0.8944724
## 2  0.8994975
## 3  0.8994975
## 4  0.9045226
## 5  0.9045226
## 6  0.9095477
## 7  0.8894472
## 8  0.8944724
## 9  0.8994975
## 10 0.8894472
## 11 0.8944724
## 12 0.8793970
## 13 0.8894472
## 14 0.8944724
## 15 0.8894472
## 16 0.8894472
## 17 0.8894472
## 18 0.8743719
## 19 0.8894472
## 20 0.8894472
```

```r
rank_plot_data <- model_comparison %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1))
  ) %>%
  select(Model_Label, AUC_Rank, F1_Rank) %>%
  pivot_longer(
    cols = c(AUC_Rank, F1_Rank),
    names_to = "Metric",
    values_to = "Rank"
  ) %>%
  mutate(
    Metric = recode(Metric,
      AUC_Rank = "AUC",
      F1_Rank = "F1"
    )
  )

ggplot(rank_plot_data, aes(x = reorder(Model_Label, Rank), y = Rank, color = Metric)) +
  geom_point(aes(size = Rank == 1)) +
  coord_flip() +
  scale_y_reverse() +
  scale_size_manual(values = c("FALSE" = 2.5, "TRUE" = 4.5), guide = "none") +
```

```r
  theme_minimal() +
  labs(
    title = "Model Rankings by AUC and F1",
    x = "Model",
    y = "Rank (1 = Best)"
  )
```



While Elastic Net and Logistic with all sentiment features achieved the highest F1 scores, Logistic Regression with VADER features achieved the highest AUC. Since AUC is the primary evaluation metric, it is preferred as it reflects better ranking performance across all thresholds.

The plot reveals a clear trade-off between AUC and F1 across models. Ridge-based models, particularly with DistilBERT features, achieve the highest AUC, indicating strong ranking performance. In contrast, LASSO and Elastic Net models with full sentiment features achieve the highest F1 scores, suggesting better balance between precision and recall. This indicates that while transformer-based sentiment features enhance ranking ability, aggregated sentiment features improve classification performance. No single model dominates across both metrics, highlighting the importance of selecting models based on the evaluation objective.

Decision Tree

Import packages

```r
# ---- 0. Install / load packages ----
pkg_needed <- c("caret", "rpart", "rattle")
for (p in pkg_needed) {
  if (!requireNamespace(p, quietly = TRUE)) {
    install.packages(p)
  }
}

#install.packages("bitops")
#install.packages("ggplot2")
#install.packages("lattice")
#install.packages("Matrix")
#install.packages("rpart.plot")
library(caret)
library(rpart)
library(rattle)
```

```
## Warning: package 'rattle' was built under R version 4.5.3

## Loading required package: bitops

##
## Attaching package: 'bitops'

## The following object is masked from 'package:Matrix':
##
##     %&%

## Rattle: A free graphical interface for data science with R.
## Version 5.6.2 Copyright (c) 2006-2023 Togaware Pty Ltd.
## Type 'rattle()' to shake, rattle, and roll your data.
```

```r
library(rpart.plot)
```

```
## Warning: package 'rpart.plot' was built under R version 4.5.3
```

Create a subset for decision tree

```r
tree_data <- df %>%
  select(
    flags_recommend_frend,
    rating_overall,
    rating_culture_values,
    rating_work_life,
    rating_compensation_benefits,
    rating_senior_leadership,
    rating_career_opportunities,
    rating_diversity_inclusion,
```

```r
    flags_business_outlook,
    flags_ceo_approval,
    employee_length,
    employee_job_quit,
    sector,
    covid_related_review,
    career_missing
  )

# Recode recommendation / outlook / CEO approval into numeric variables
tree_data <- tree_data %>%
  dplyr::mutate(
    employee_job_quit = as.integer(employee_job_quit),
    career_missing = as.integer(career_missing),
    flags_recommend_frend = as.factor(flags_recommend_frend),
    sector = as.integer(as.factor(sector)),
    flags_business_outlook = as.integer(as.factor(flags_business_outlook)),
    flags_ceo_approval = as.integer(as.factor(flags_ceo_approval))
  )
```

Split the dataset into training and testing

```r
library(caret)

set.seed(123)

train_index <- createDataPartition(
  tree_data$flags_recommend_frend,
  p = 0.8,
  list = FALSE
)

tree.train.data <- tree_data[train_index, ]
tree.test.data <- tree_data[-train_index, ]

# Check class imbalance after splitting
prop.table(table(tree.train.data$flags_recommend_frend))
```

```
##
##  NEGATIVE  POSITIVE
## 0.6916355 0.3083645
```

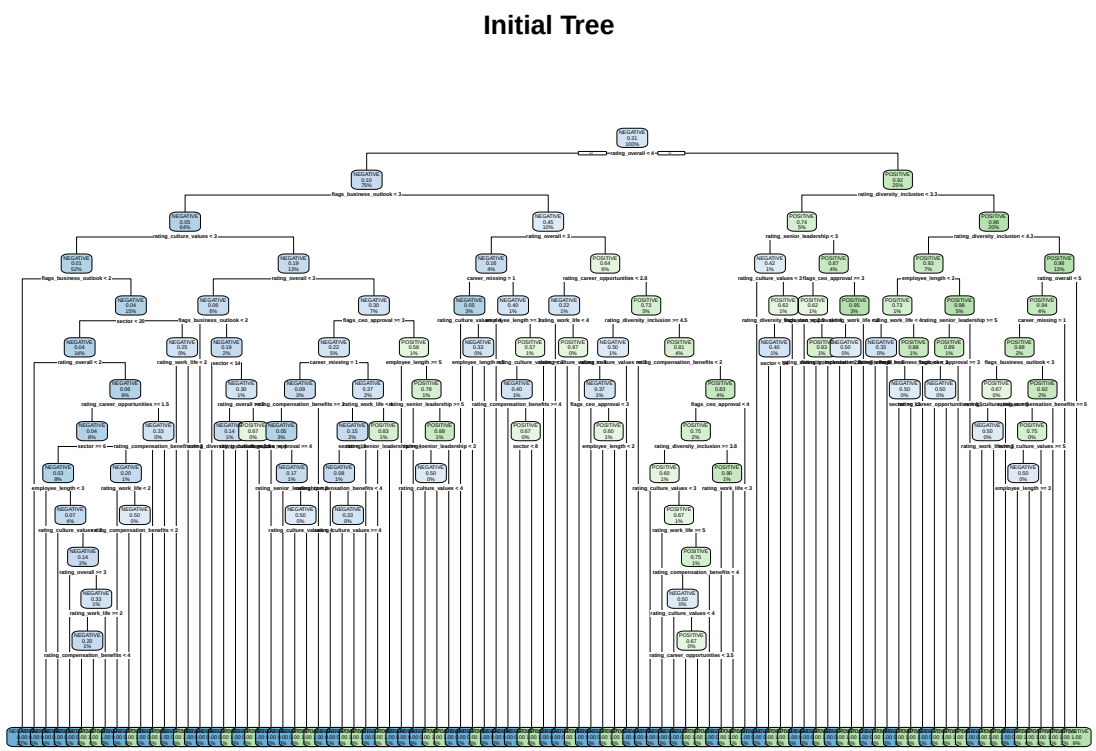
```r
prop.table(table(tree.test.data$flags_recommend_frend))
```

```
##
##  NEGATIVE  POSITIVE
## 0.6934673 0.3065327
```

Train the Decision Tree

```
# Fit the initial tree
set.seed(123)
tree_model <- rpart(
  flags_recommend_frend ~ .,
  data = tree.train.data,
  method = "class",
  control = rpart.control(cp = 0, xval = 10, minsplit = 2, minbucket = 1)
)

# Visualize the tree
rpart.plot(tree_model, main = "Initial Tree")
```

```
## Warning: labs do not fit even at cex 0.15, there may be some overplotting
```




```
# Check CP table
printcp(tree_model)
```

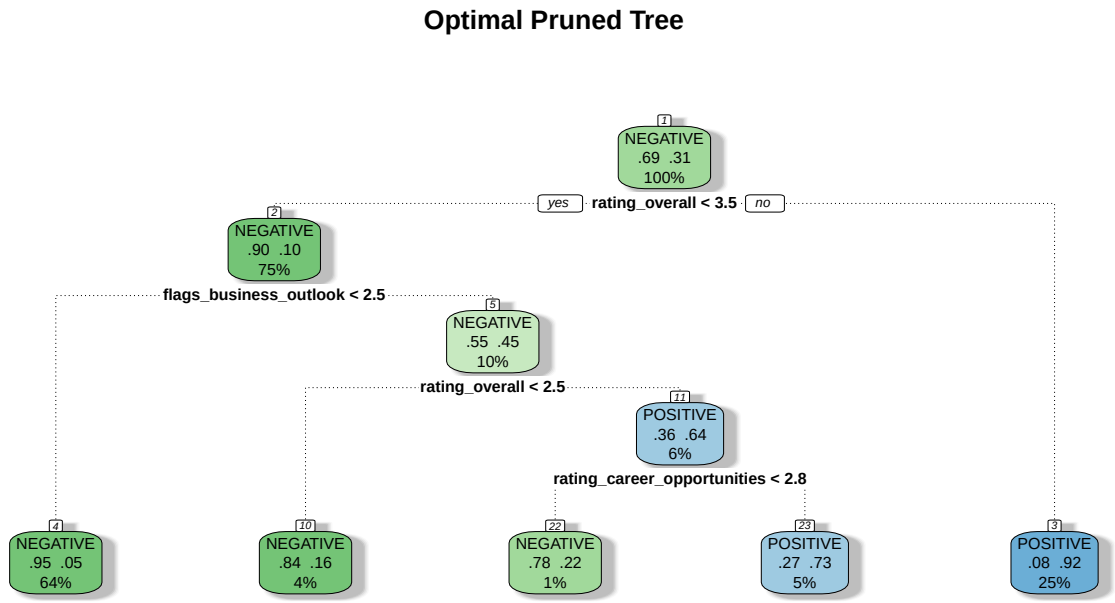
```
##
## Classification tree:
## rpart(formula = flags_recommend_frend ~ ., data = tree.train.data,
##     method = "class", control = rpart.control(cp = 0, xval = 10,
```

```
##         minsplit = 2, minbucket = 1))
##
## Variables actually used in tree construction:
##  [1] career_missing               employee_length
##  [3] flags_business_outlook       flags_ceo_approval
##  [5] rating_career_opportunities  rating_compensation_benefits
##  [7] rating_culture_values        rating_diversity_inclusion
##  [9] rating_overall               rating_senior_leadership
## [11] rating_work_life             sector
##
## Root node error: 247/801 = 0.30836
##
## n= 801
##
##           CP nsplit rel error  xerror     xstd
## 1  0.6842105      0 1.0000000 1.00000 0.052916
## 2  0.0283401      1 0.3157895 0.31579 0.033971
## 3  0.0202429      3 0.2591093 0.29960 0.033180
## 4  0.0053981      4 0.2388664 0.25506 0.030845
## 5  0.0050607      9 0.2105263 0.28745 0.032567
## 6  0.0040486     15 0.1740891 0.28745 0.032567
## 7  0.0032389     29 0.1133603 0.34008 0.035106
## 8  0.0024291     34 0.0971660 0.36032 0.036010
## 9  0.0020243     47 0.0647773 0.40486 0.037874
## 10 0.0013495     61 0.0364372 0.40891 0.038036
## 11 0.0010121     81 0.0080972 0.43725 0.039135
## 12 0.0000000     89 0.0000000 0.44130 0.039287
```

```r
# Select the best CP value
best_cp <- tree_model$cptable[
  which.min(tree_model$cptable[, "xerror"]),
  "CP"
]

cat("\nBest CP value:", best_cp, "\n")
```

```
##
## Best CP value: 0.005398111
```

```r
# Prune the tree using best CP value
pruned_tree <- prune(tree_model, cp = best_cp)

# Visualize the pruned tree
fancyRpartPlot(pruned_tree, main = "Optimal Pruned Tree")
```




```r
# Predict on test data
pred_test <- predict(pruned_tree, newdata = tree.test.data, type = "class")

# Confusion Matrix and Metrics
pruned_tree_cm <- confusionMatrix(pred_test, tree.test.data$flags_recommend_frend)
pruned_tree_cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      126       13
##   POSITIVE       12       48
##
##                Accuracy : 0.8744
##                  95% CI : (0.8202, 0.917)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.833e-09
##
##                   Kappa : 0.7031
##
##  Mcnemar's Test P-Value : 1
##
##             Sensitivity : 0.9130
##             Specificity : 0.7869
##          Pos Pred Value : 0.9065
```

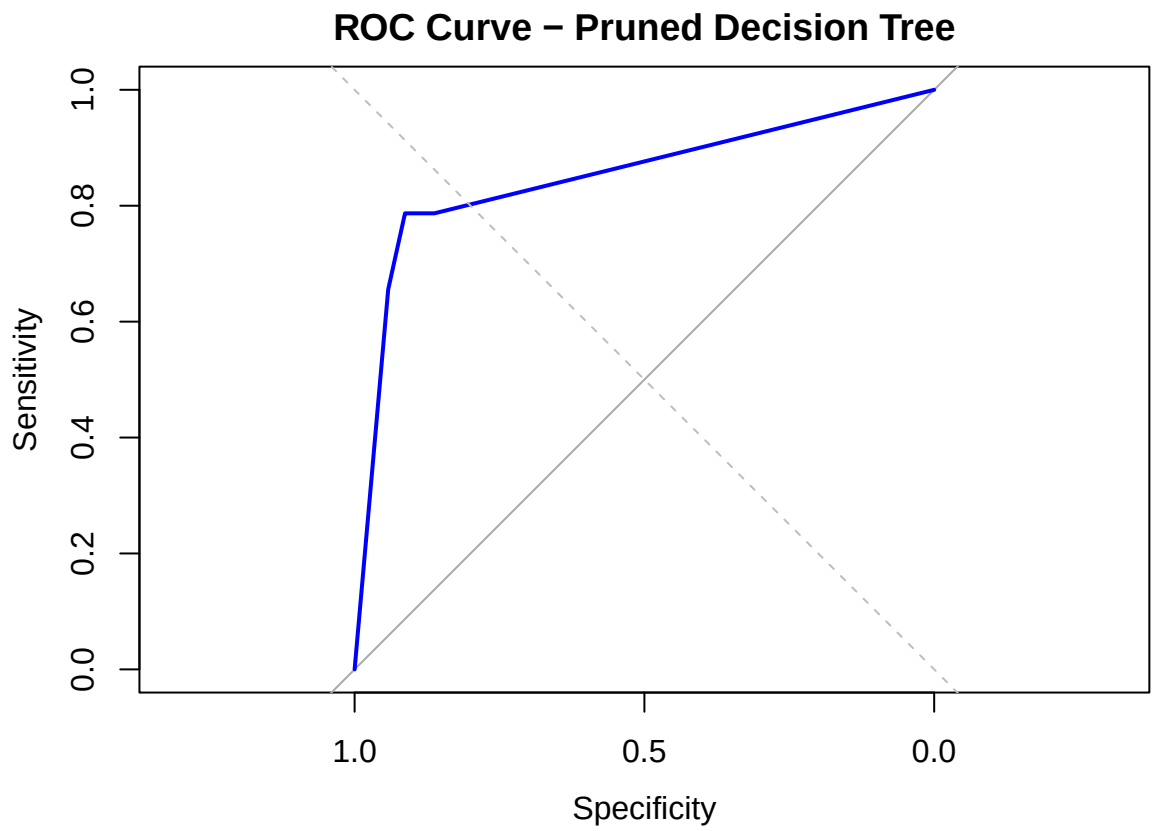
```
##          Neg Pred Value : 0.8000
##              Prevalence : 0.6935
##          Detection Rate : 0.6332
##    Detection Prevalence : 0.6985
##       Balanced Accuracy : 0.8500
##
##        'Positive' Class : NEGATIVE
##
```

ROC curve and AUC

```r
# ROC and AUC for pruned tree
#install.packages("pROC")
library(pROC)

# Make sure outcome levels are ordered correctly
tree.test.data$flags_recommend_frend <- factor(
  tree.test.data$flags_recommend_frend,
  levels = c("NEGATIVE", "POSITIVE")
)

# Predicted probabilities
pred_prob <- predict(pruned_tree, newdata = tree.test.data, type = "prob")

# Probability of positive class
pred_prob_positive <- pred_prob[, "POSITIVE"]

# ROC curve
roc_pruned_tree <- roc(
  response = tree.test.data$flags_recommend_frend,
  predictor = pred_prob_positive,
  levels = c("NEGATIVE", "POSITIVE"),
  direction = "<"
)

# AUC
auc_pruned_tree <- auc(roc_pruned_tree)
auc_pruned_tree
```

```
## Area under the curve: 0.8503
```

```r
# Plot ROC
plot(
  roc_pruned_tree,
  col = "blue",
  lwd = 2,
  main = "ROC Curve - Pruned Decision Tree"
)

# Add diagonal reference line
abline(a = 0, b = 1, lty = 2, col = "gray")
```



Random forest:

Import packages

```
#install.packages("randomForest")
#install.packages("ggplot2")
library(randomForest)
```

```
## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:rattle':
##
##     importance

## The following object is masked from 'package:dplyr':
##
##     combine
```

```
## The following object is masked from 'package:ggplot2':
##
##     margin
```

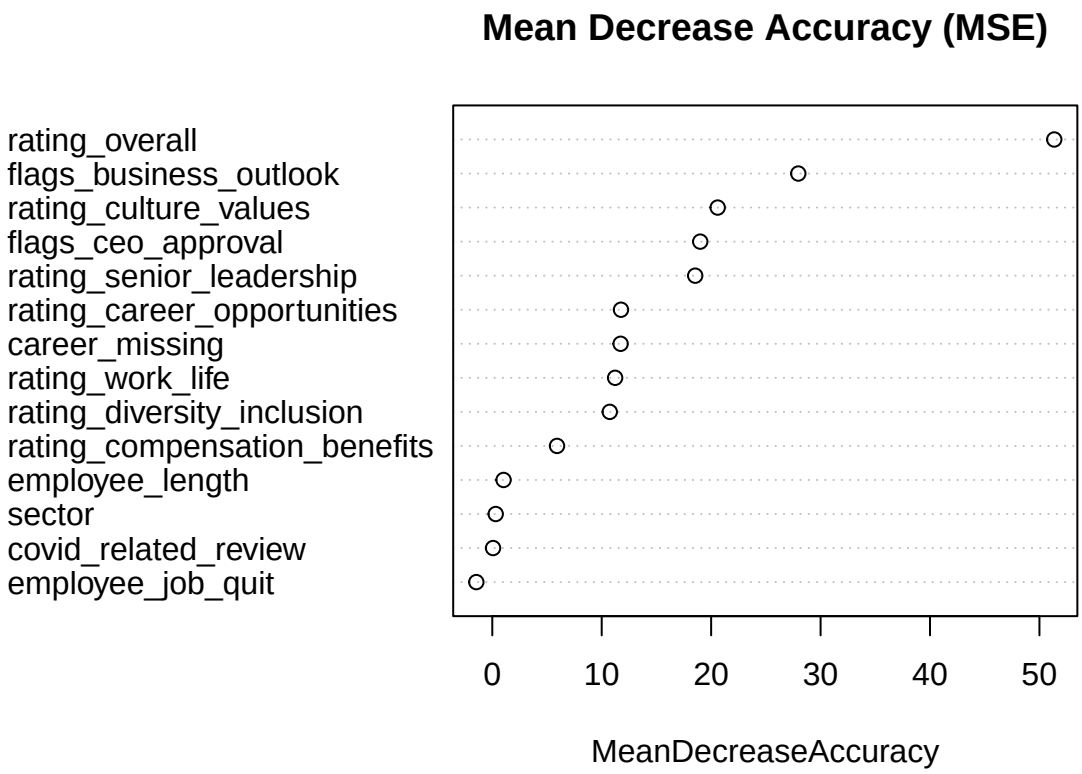
```r
library(caret)
```

Train the random forest

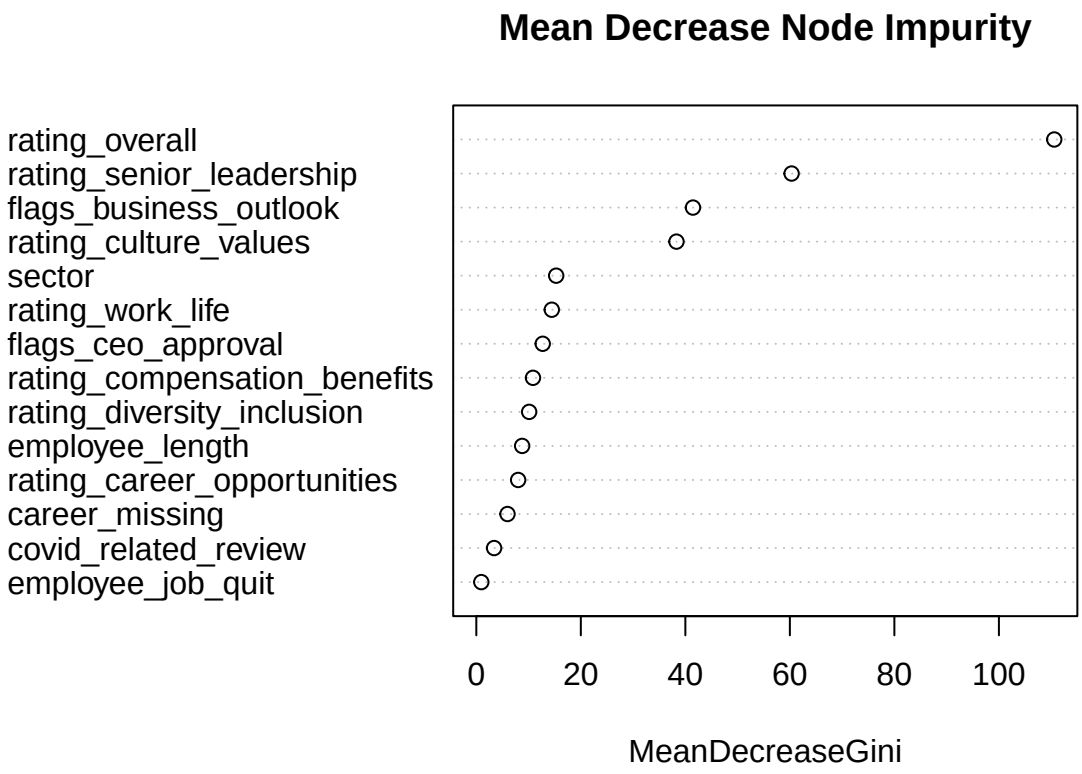
```r
# Train Random Forest with 10-fold CV
set.seed(123)

p <- ncol(tree.train.data) - 1

rf_model <- train(
  flags_recommend_frend ~ .,
  data = tree.train.data,
  method = "rf",
  trControl = trainControl(method = "cv", number = 10),
  tuneGrid = expand.grid(mtry = 1:p),
  ntree = 500,
  importance = TRUE
)

# Find the best mtry
bestTune <- rf_model$bestTune
bestTune
```

```
##   mtry
## 6    6
```

```r
# Refit the model with the best mtry
rf_model_tuned <- train(
  flags_recommend_frend ~ .,
  data = tree.train.data,
  method = "rf",
  trControl = trainControl(
    method = "cv",
    number = 10,
    classProbs = TRUE,
    summaryFunction = twoClassSummary
  ),
  tuneGrid = expand.grid(mtry = bestTune),  # use your chosen mtry
  ntree = 500,
  metric = "ROC",
  importance = TRUE
)

# Variable importance plots
varImpPlot(rf_model_tuned$finalModel, type = 1, main = "Mean Decrease Accuracy (MSE)")  # Mean Decrease
```




```
varImpPlot(rf_model_tuned$finalModel, type = 2, main = "Mean Decrease Node Impurity")  # Mean Decrease
```




```r
# Predict on test data
pred_test <- predict(rf_model_tuned, tree.test.data)

# Confusion matrix and metrics
cm <- confusionMatrix(
  pred_test,
  tree.test.data$flags_recommend_frend
)

cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      130       16
##   POSITIVE        8       45
##
##                Accuracy : 0.8794
##                  95% CI : (0.8259, 0.9212)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.802e-10
##
##                   Kappa : 0.7055
##
##  Mcnemar's Test P-Value : 0.153
```

```
##
##             Sensitivity : 0.9420
##             Specificity : 0.7377
##          Pos Pred Value : 0.8904
##          Neg Pred Value : 0.8491
##              Prevalence : 0.6935
##          Detection Rate : 0.6533
##    Detection Prevalence : 0.7337
##       Balanced Accuracy : 0.8399
##
##        'Positive' Class : NEGATIVE
##
```

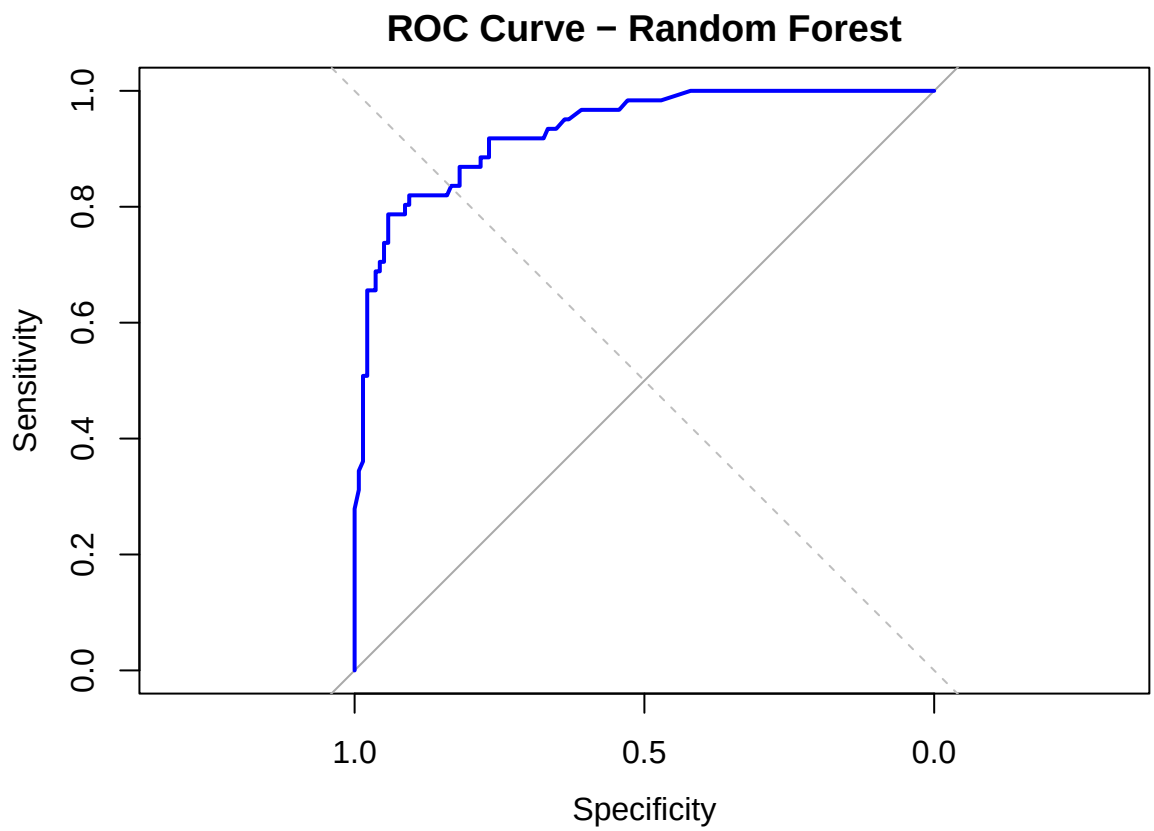
```r
# Metrics
sensitivity <- cm$byClass["Sensitivity"]
specificity <- cm$byClass["Specificity"]
accuracy    <- cm$overall["Accuracy"]

#sensitivity
#specificity
#accuracy
```

ROC curve and AUC

```r
library(pROC)
# Predict probabilities
rf_pred_prob <- predict(rf_model_tuned, newdata = tree.test.data, type = "prob")

# Extract probability of POSITIVE class
rf_pred_prob_pos <- rf_pred_prob[, "POSITIVE"]

# Make Sure Target is Factor
test.data$flags_recommend_frend <- factor(
  tree.test.data$flags_recommend_frend,
  levels = c("NEGATIVE", "POSITIVE")
)

# Compute ROC Curve
roc_rf <- roc(
  response = tree.test.data$flags_recommend_frend,
  predictor = rf_pred_prob_pos,
  levels = c("NEGATIVE", "POSITIVE"),
  direction = "<"
)

# AUC
auc_rf <- auc(roc_rf)
auc_rf
```

```
## Area under the curve: 0.9308
```

```r
# Plot ROC curve
plot(
  roc_rf,
  col = "blue",
  lwd = 2,
  main = "ROC Curve - Random Forest"
)

abline(a = 0, b = 1, lty = 2, col = "gray")
```



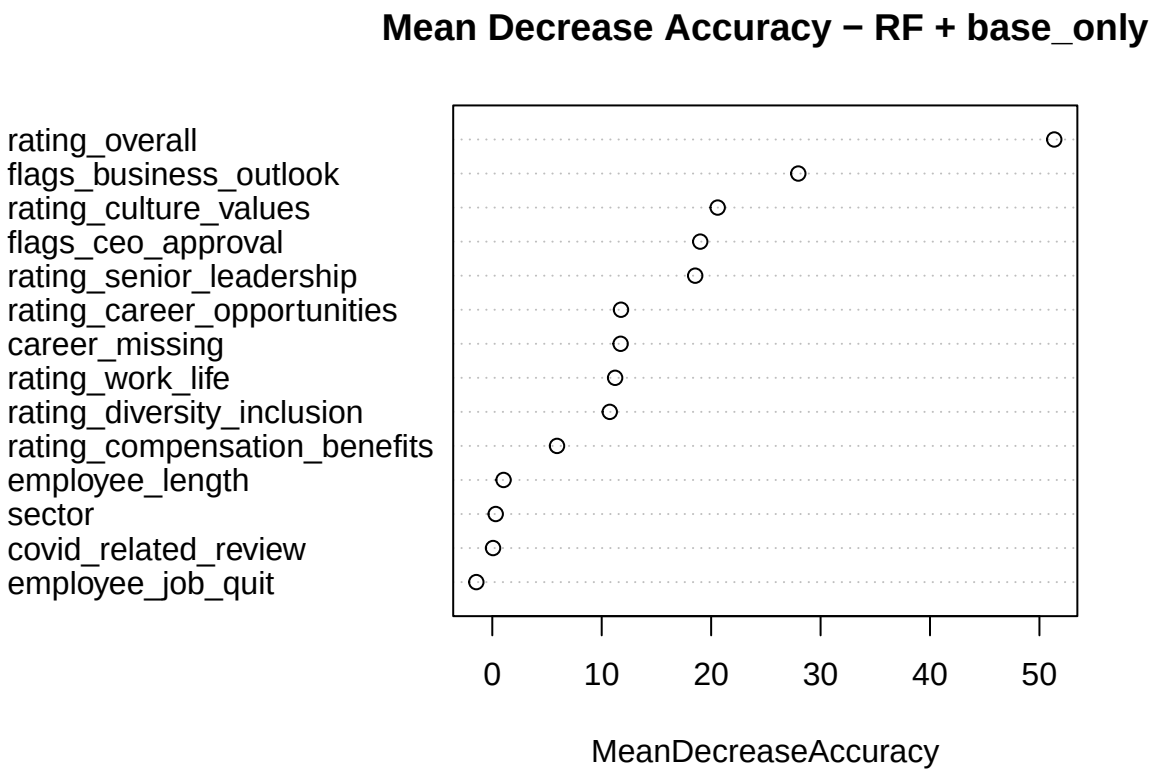
RF Sentiment Add-ons

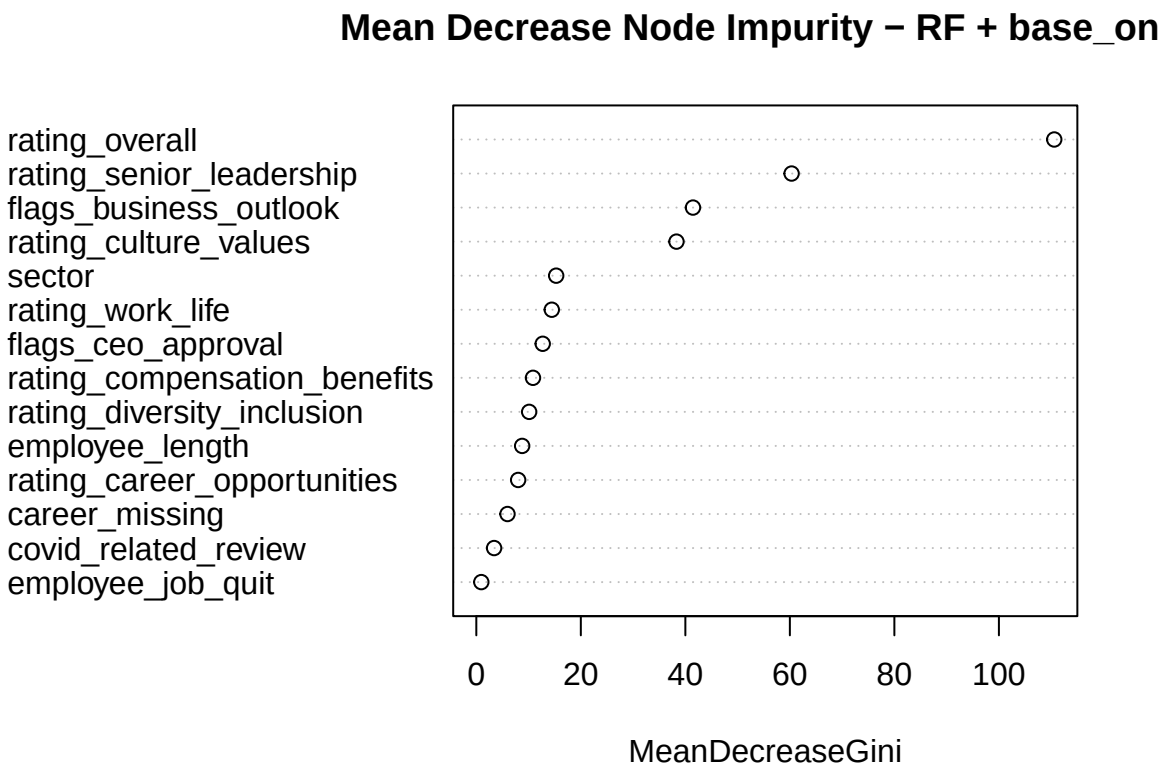
```r
tree_datasets <- list(
  base_only = tree_data,
  vader = cbind(tree_data, df[, c("sent_vader_avg", "sent_vader_sd")]),
  distilbert = cbind(tree_data, df[, c("sent_distilbert_avg", "sent_distilbert_sd")]),
  roberta = cbind(tree_data, df[, c("sent_roberta_avg", "sent_roberta_sd")]),
  base_and_all_sent = cbind(tree_data, df[, c(
    "sent_vader_avg", "sent_vader_sd",
    "sent_distilbert_avg", "sent_distilbert_sd",
    "sent_roberta_avg", "sent_roberta_sd"
  )])
)
```

Looping through models and building up result list

```
rf_results <- list()

for (name in names(tree_datasets)) {

  df_current <- tree_datasets[[name]]

  train_df <- df_current[train_index, ]
  test_df  <- df_current[-train_index, ]

  set.seed(123)

  # Number of predictors for current dataset
  p <- ncol(train_df) - 1

  # Step 1: Tune mtry over full grid
  rf_model <- train(
    flags_recommend_frend ~ .,
    data = train_df,
    method = "rf",
    trControl = trainControl(method = "cv", number = 10),
    tuneGrid = expand.grid(mtry = 1:p),
    ntree = 500,
    importance = TRUE
  )

  # Best mtry for this dataset
  bestTune <- rf_model$bestTune

  # Step 2: Refit with best mtry using ROC
  rf_model_tuned <- train(
    flags_recommend_frend ~ .,
    data = train_df,
    method = "rf",
    trControl = trainControl(
      method = "cv",
      number = 10,
      classProbs = TRUE,
      summaryFunction = twoClassSummary
    ),
    tuneGrid = bestTune,
    ntree = 500,
    metric = "ROC",
    importance = TRUE
  )

  # Optional: variable importance plots
  # Keep this if you want plots for every dataset; otherwise remove or restrict
  varImpPlot(
    rf_model_tuned$finalModel,
    type = 1,
    main = paste("Mean Decrease Accuracy - RF +", name)
```

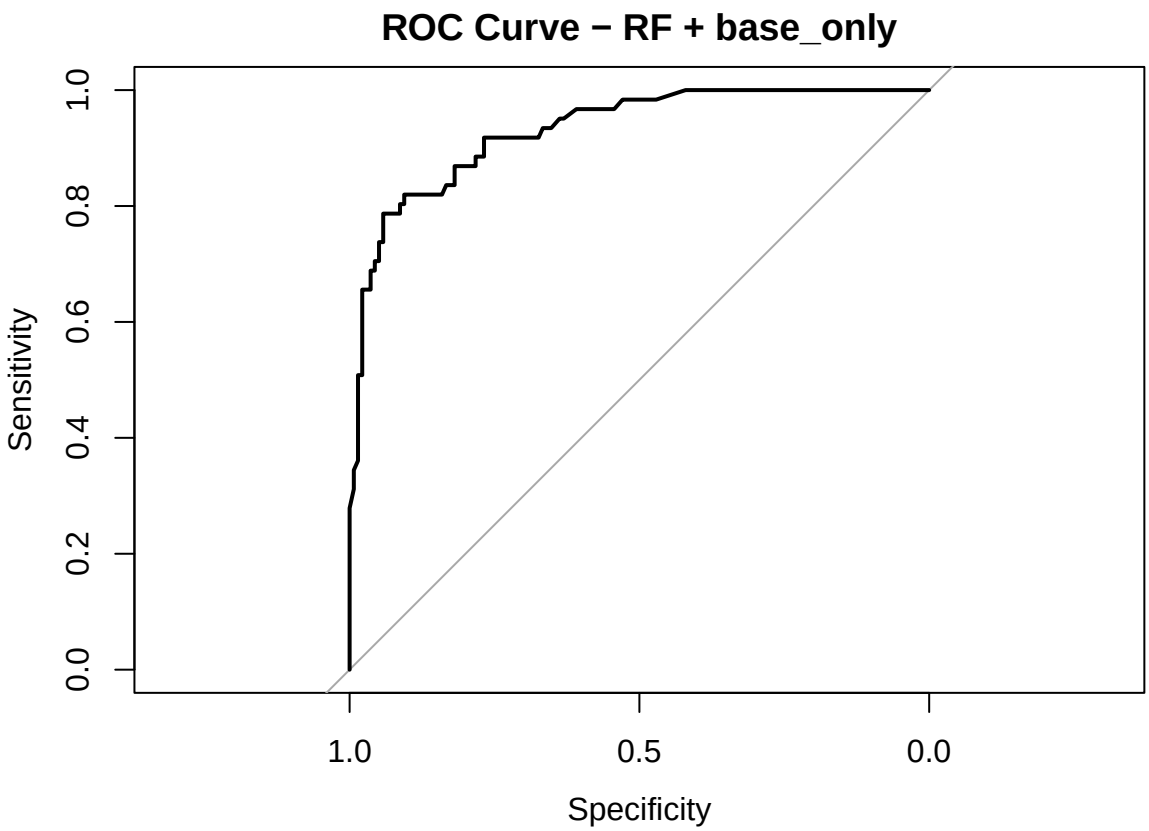
```r
  )

  varImpPlot(
    rf_model_tuned$finalModel,
    type = 2,
    main = paste("Mean Decrease Node Impurity - RF +", name)
  )

  # Predict probabilities for POSITIVE class
  prob <- predict(rf_model_tuned, newdata = test_df, type = "prob")[, "POSITIVE"]

  # Evaluate
  eval <- evaluate_model(
    prob = prob,
    actual = test_df$flags_recommend_frend,
    model_name = paste("RF +", name)
  )

  # Store everything
  rf_results[[name]] <- list(
    best_mtry = bestTune$mtry,
    rf_model = rf_model,
    rf_model_tuned = rf_model_tuned,
    eval = eval
  )
}
```


Mean Decrease Accuracy − RF + base_only
rating_overall
flags_business_outlook
rating_culture_values
flags_ceo_approval
rating_senior_leadership
rating_career_opportunities
career_missing
rating_work_life
rating_diversity_inclusion
rating_compensation_benefits
employee_length
sector
covid_related_review
employee_job_quit
0
10
20
30
40
50
MeanDecreaseAccuracy

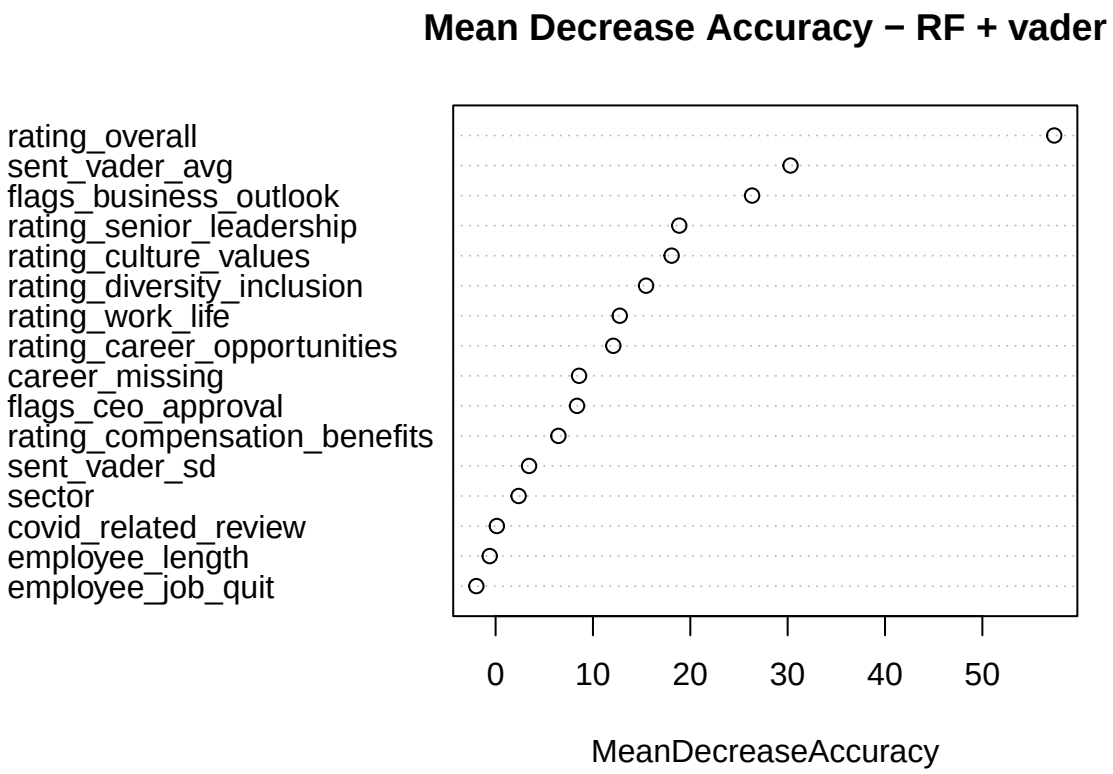


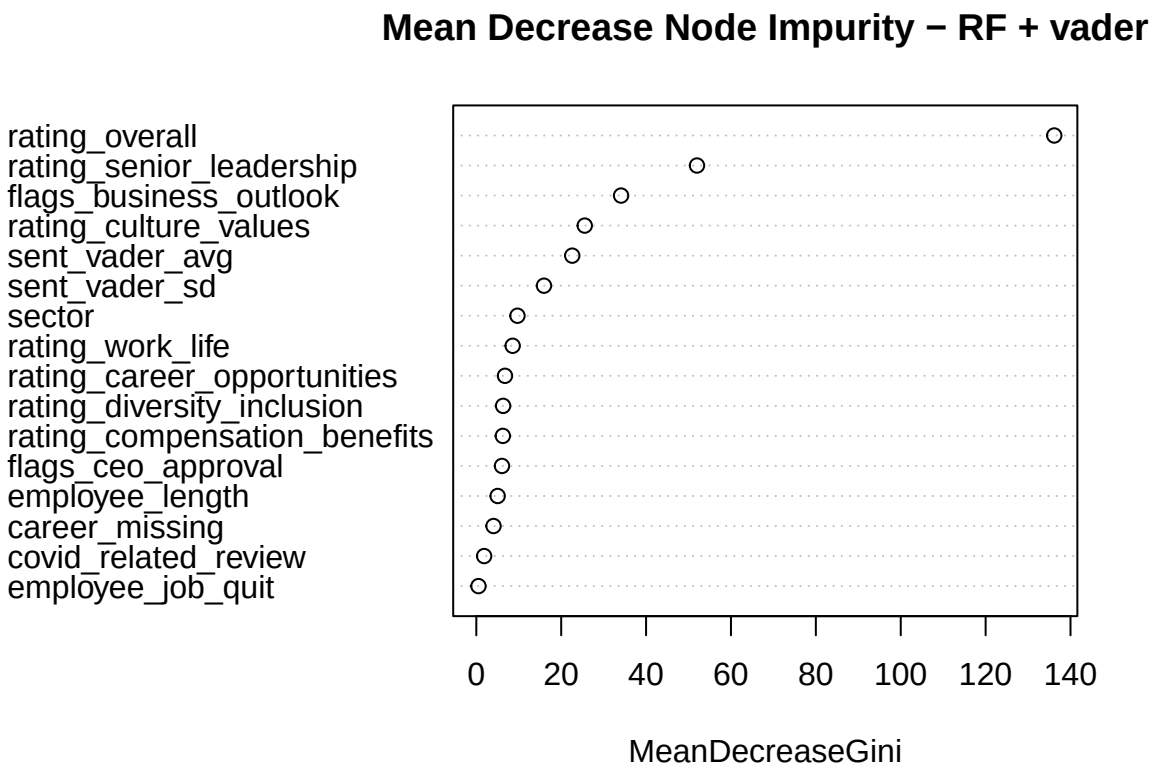

```
## Best threshold for RF + base_only : 0.45
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      130       13
##   POSITIVE        8       48
##
##                Accuracy : 0.8945
##                  95% CI : (0.8432, 0.9335)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.388e-11
##
##                   Kappa : 0.746
##
##  Mcnemar's Test P-Value : 0.3827
##
##             Sensitivity : 0.9420
##             Specificity : 0.7869
##          Pos Pred Value : 0.9091
##          Neg Pred Value : 0.8571
##              Prevalence : 0.6935
##          Detection Rate : 0.6533
##    Detection Prevalence : 0.7186
##       Balanced Accuracy : 0.8645
##
```

```
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE

## Setting direction: controls < cases
```



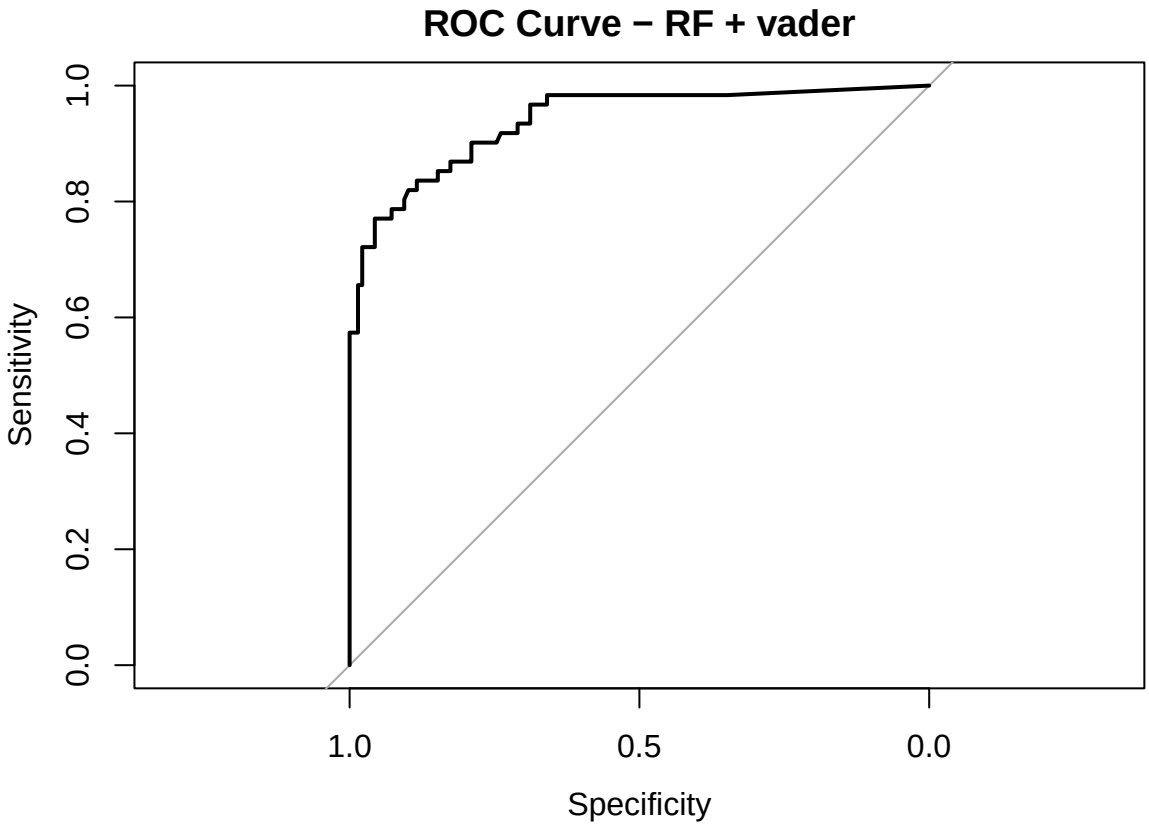

```
## RF + base_only AUC: 0.930803
```


Mean Decrease Accuracy − RF + vader
rating_overall
sent_vader_avg
flags_business_outlook
rating_senior_leadership
rating_culture_values
rating_diversity_inclusion
rating_work_life
rating_career_opportunities
career_missing
flags_ceo_approval
rating_compensation_benefits
sent_vader_sd
sector
covid_related_review
employee_length
employee_job_quit
0
10
20
30
40
50
MeanDecreaseAccuracy

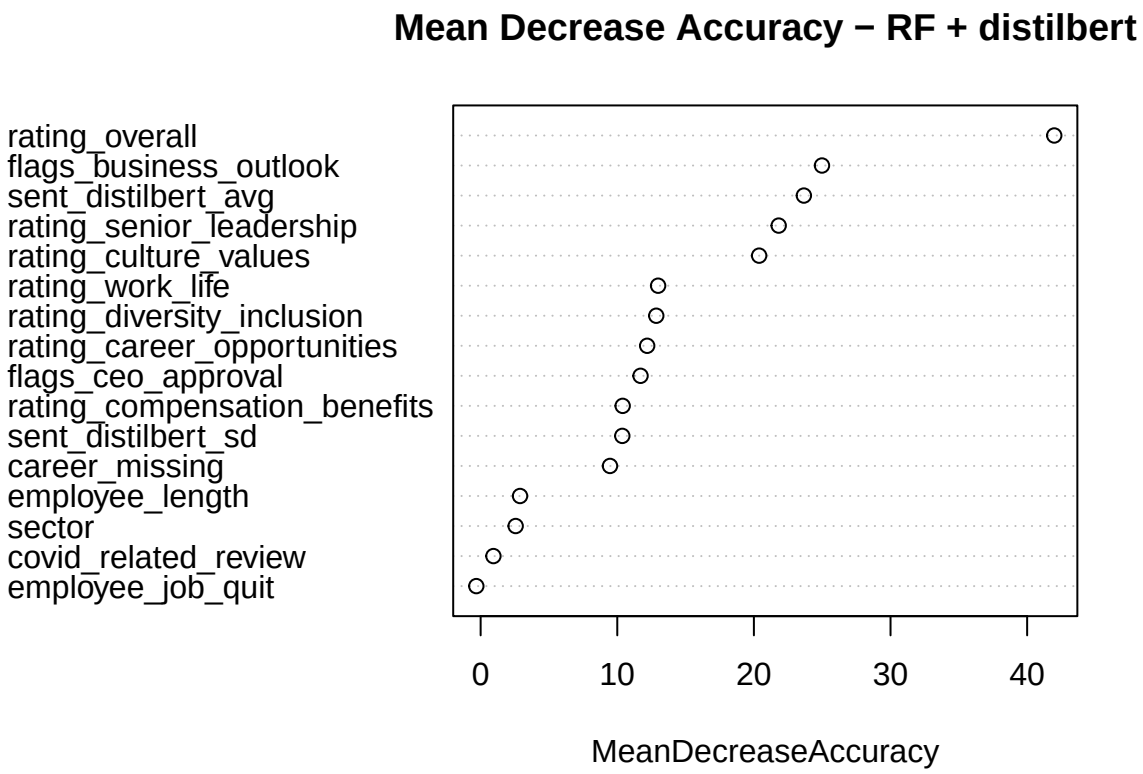


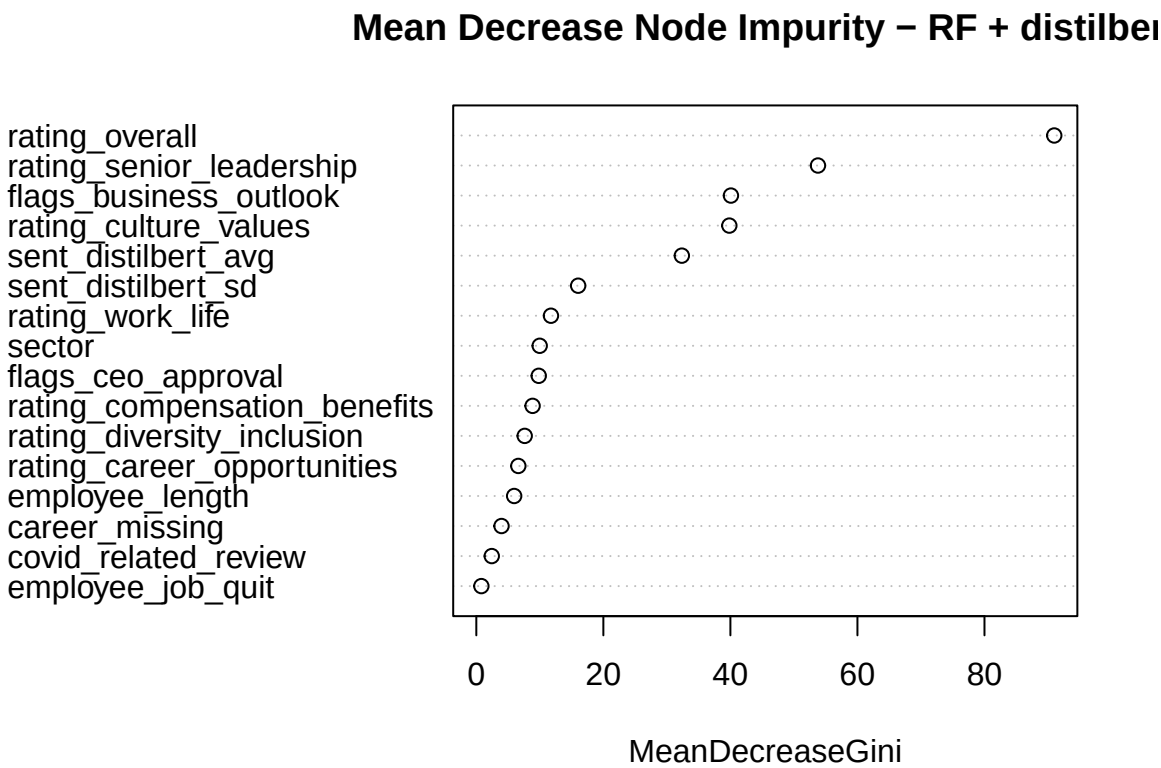

```
## Best threshold for RF + vader : 0.55
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      132       14
##   POSITIVE        6       47
##
##                Accuracy : 0.8995
##                  95% CI : (0.8491, 0.9375)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 3.617e-12
##
##                   Kappa : 0.7546
##
##  Mcnemar's Test P-Value : 0.1175
##
##             Sensitivity : 0.9565
##             Specificity : 0.7705
##          Pos Pred Value : 0.9041
##          Neg Pred Value : 0.8868
##              Prevalence : 0.6935
##          Detection Rate : 0.6633
##    Detection Prevalence : 0.7337
##       Balanced Accuracy : 0.8635
##
```

```
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```



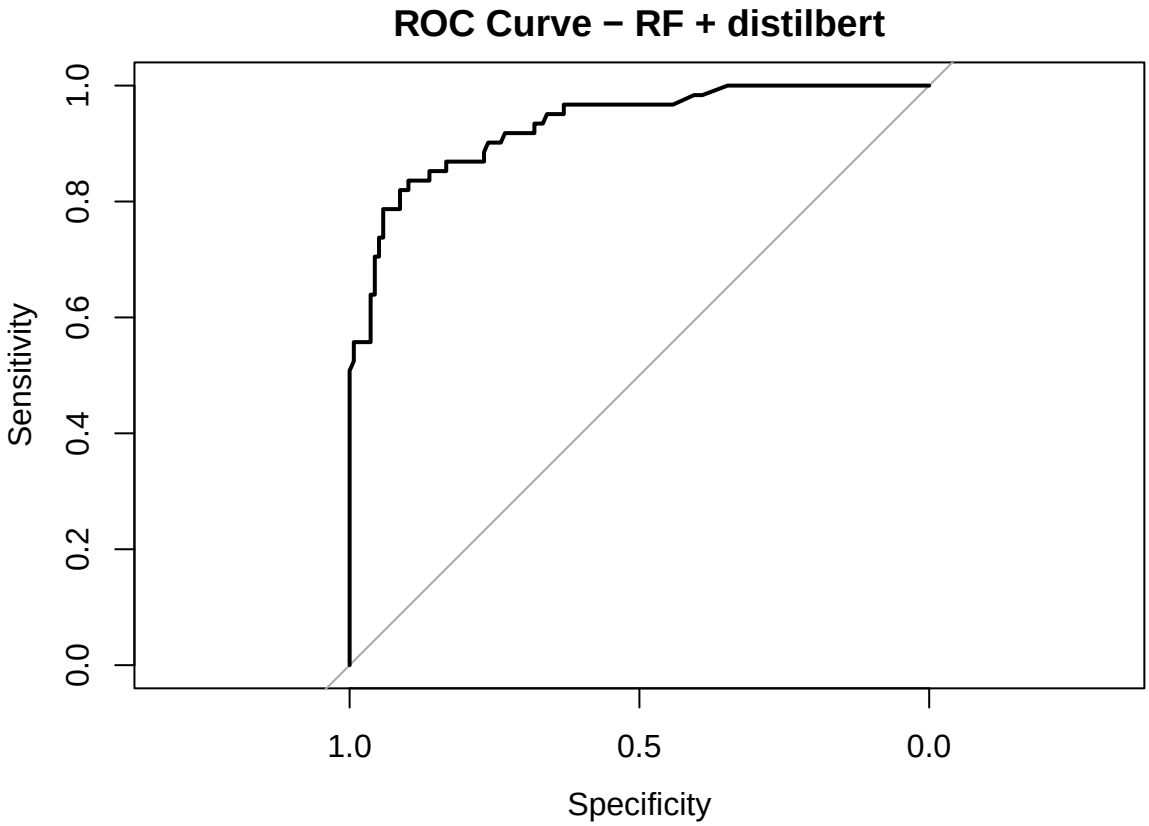

```
## RF + vader AUC: 0.9384652
```


Mean Decrease Accuracy − RF + distilbert
rating_overall
flags_business_outlook
sent_distilbert_avg
rating_senior_leadership
rating_culture_values
rating_work_life
rating_diversity_inclusion
rating_career_opportunities
flags_ceo_approval
rating_compensation_benefits
sent_distilbert_sd
career_missing
employee_length
sector
covid_related_review
employee_job_quit
0
10
20
30
40
MeanDecreaseAccuracy

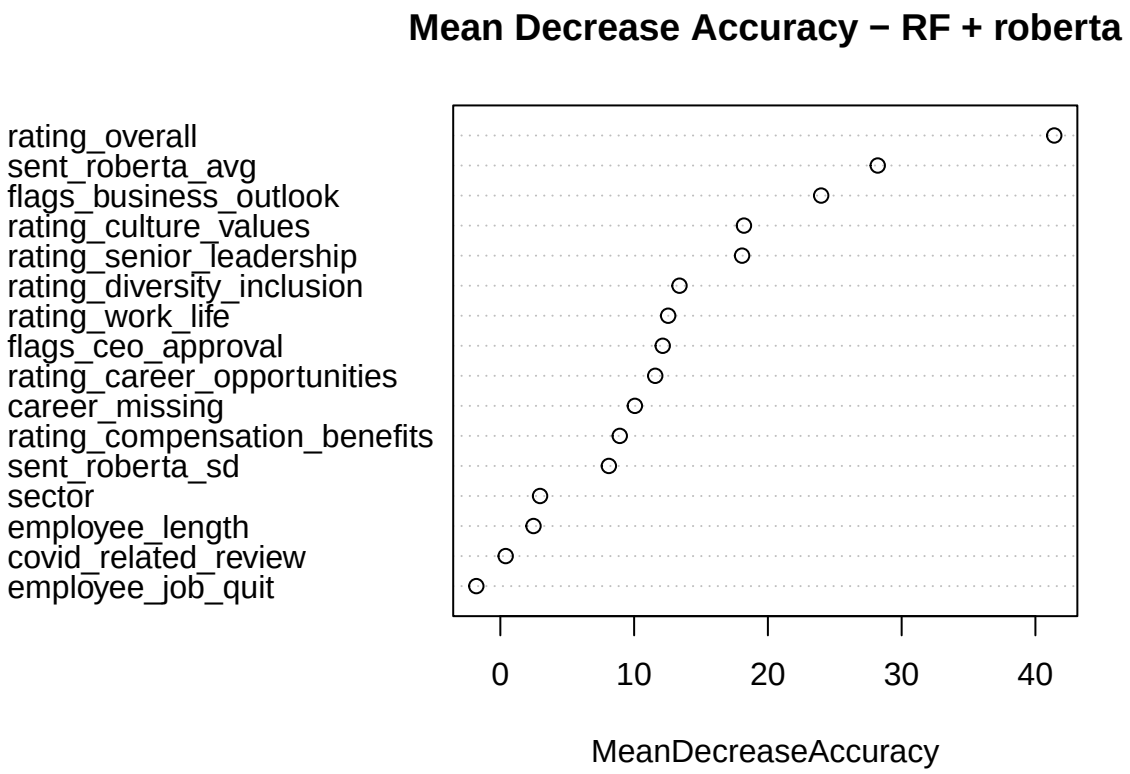


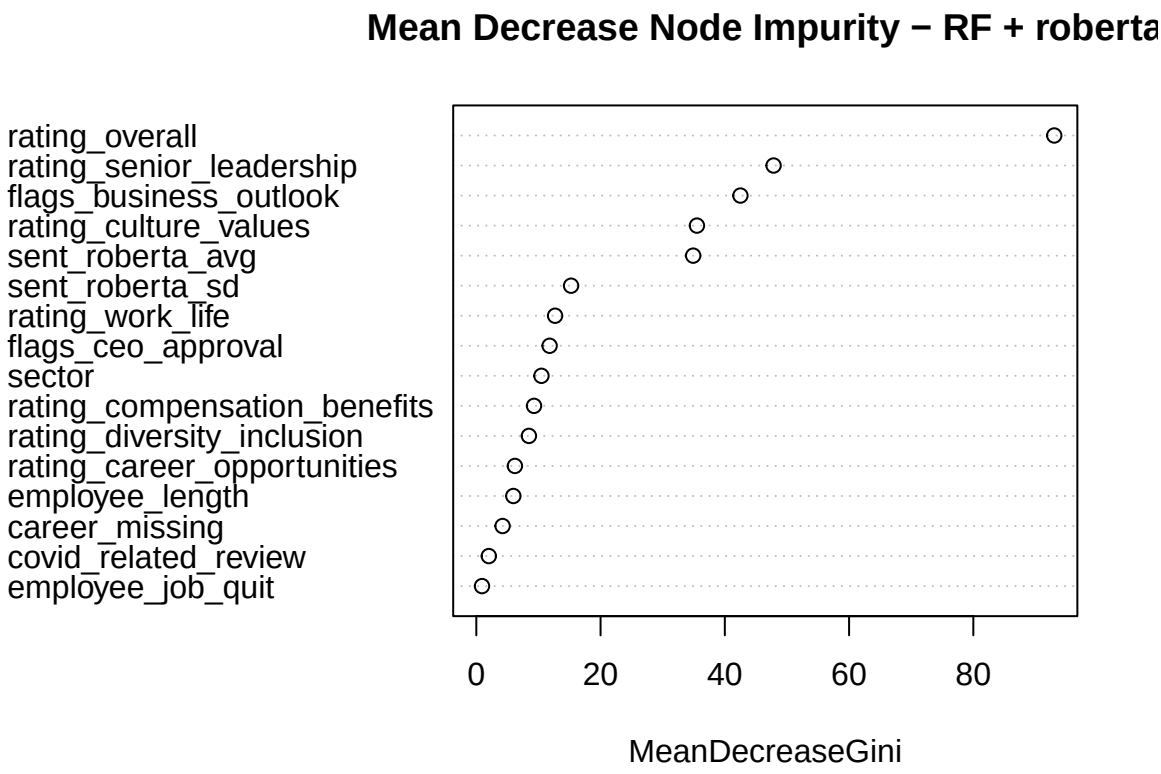

```
## Best threshold for RF + distilbert : 0.45
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      130       14
##   POSITIVE        8       47
##
##                Accuracy : 0.8894
##                  95% CI : (0.8374, 0.9294)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 5.059e-11
##
##                   Kappa : 0.7326
##
##  Mcnemar's Test P-Value : 0.2864
##
##             Sensitivity : 0.9420
##             Specificity : 0.7705
##          Pos Pred Value : 0.9028
##          Neg Pred Value : 0.8545
##              Prevalence : 0.6935
##          Detection Rate : 0.6533
##    Detection Prevalence : 0.7236
##       Balanced Accuracy : 0.8563
##
```

```
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```



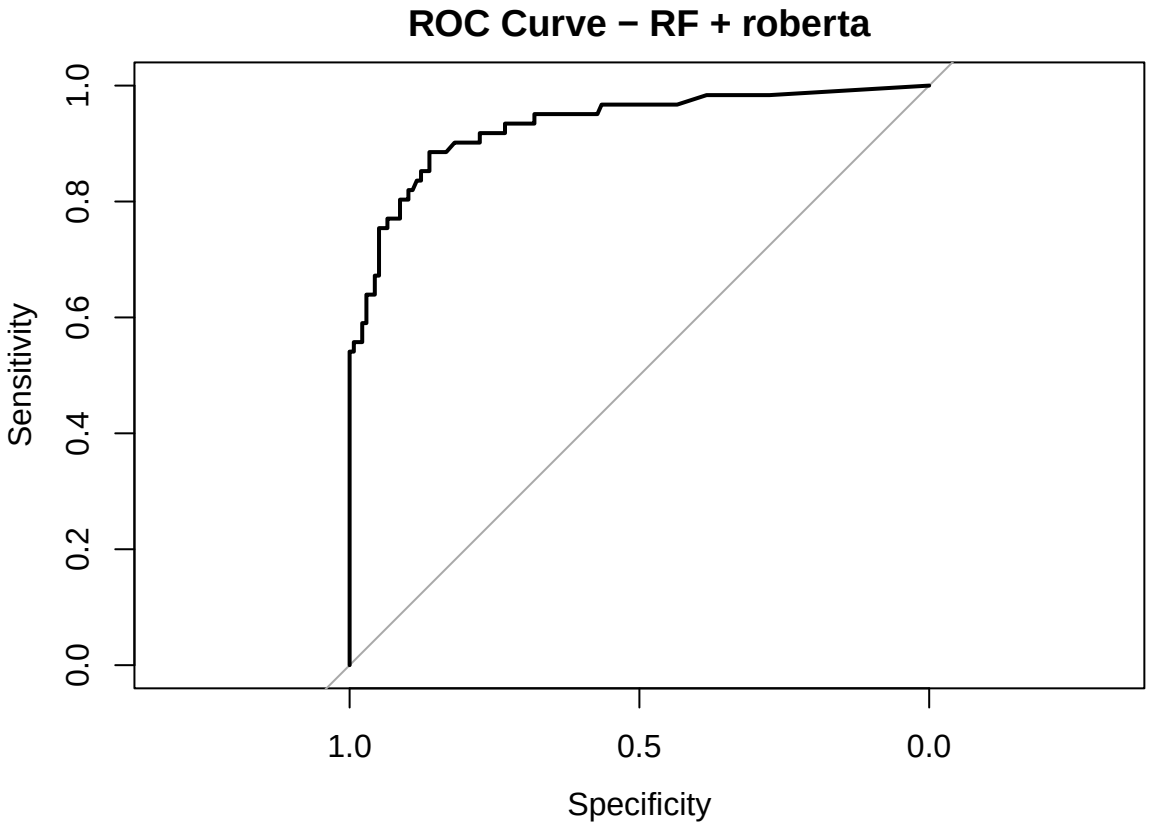

```
## RF + distilbert AUC: 0.9312782
```


Mean Decrease Accuracy − RF + roberta
rating_overall
sent_roberta_avg
flags_business_outlook
rating_culture_values
rating_senior_leadership
rating_diversity_inclusion
rating_work_life
flags_ceo_approval
rating_career_opportunities
career_missing
rating_compensation_benefits
sent_roberta_sd
sector
employee_length
covid_related_review
employee_job_quit
0
10
20
30
40
MeanDecreaseAccuracy

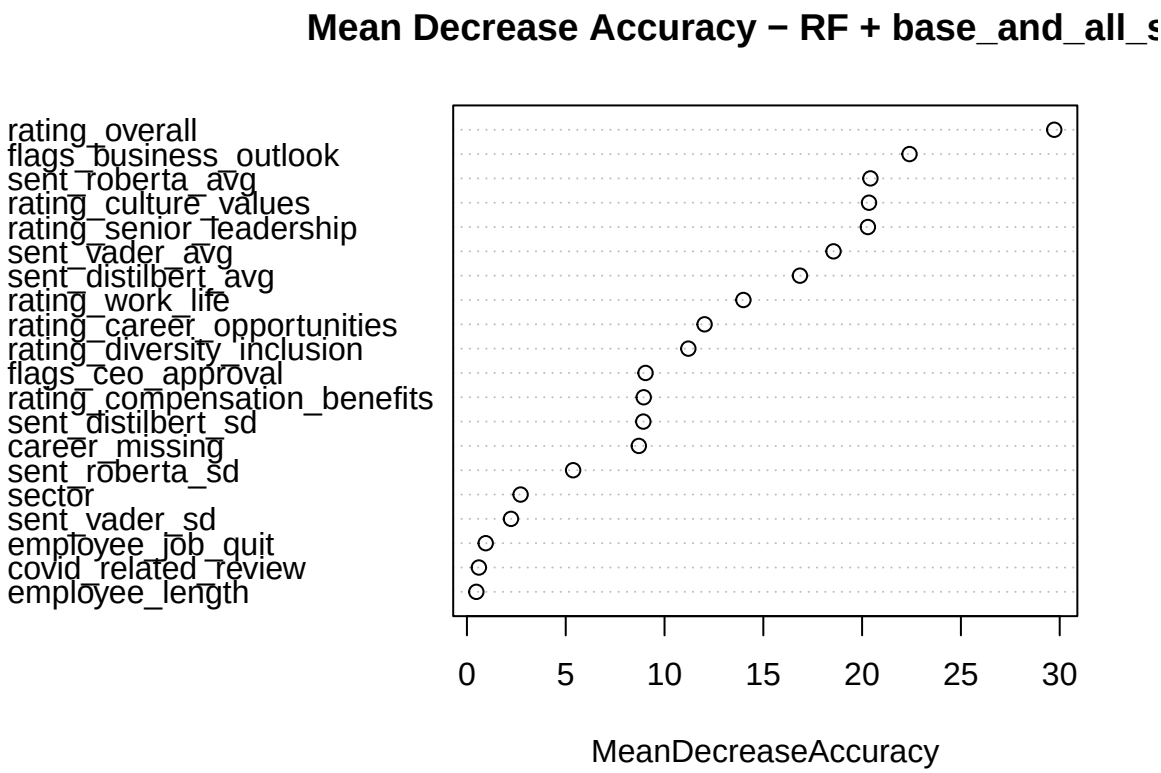


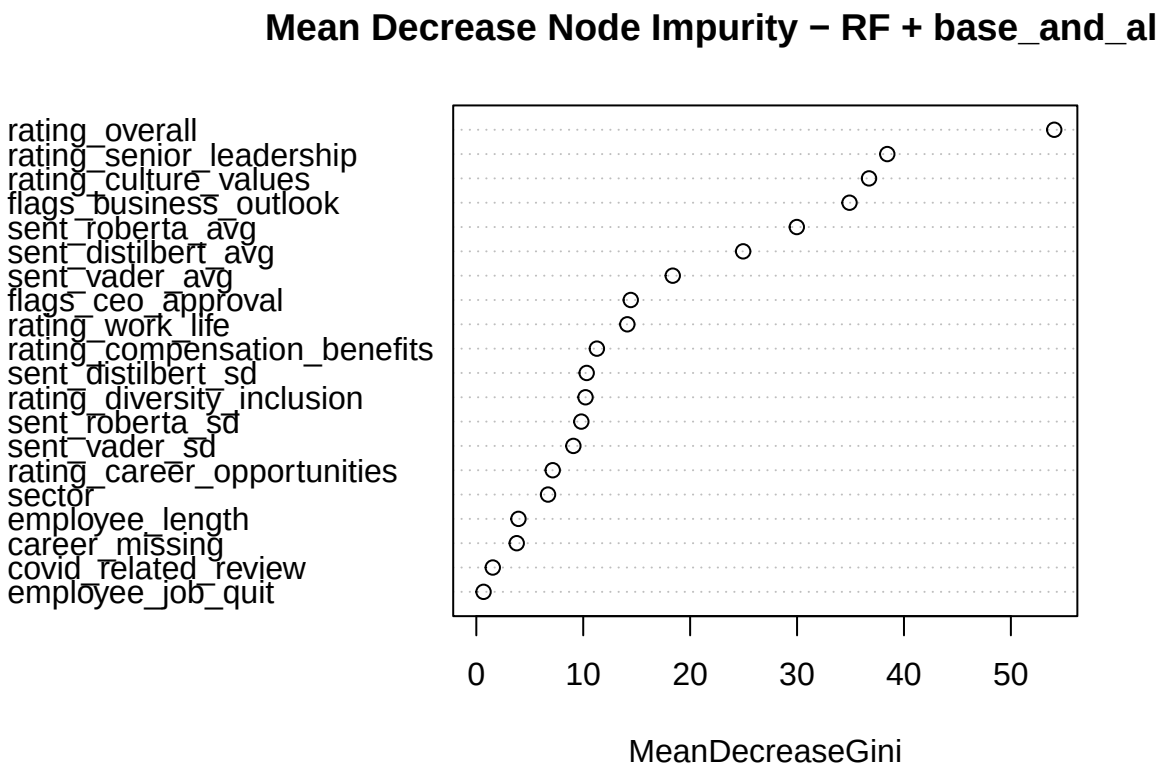

```
## Best threshold for RF + roberta : 0.5
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      130       15
##   POSITIVE        8       46
##
##                Accuracy : 0.8844
##                  95% CI : (0.8316, 0.9253)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.755e-10
##
##                   Kappa : 0.7192
##
##  Mcnemar's Test P-Value : 0.2109
##
##             Sensitivity : 0.9420
##             Specificity : 0.7541
##          Pos Pred Value : 0.8966
##          Neg Pred Value : 0.8519
##              Prevalence : 0.6935
##          Detection Rate : 0.6533
##    Detection Prevalence : 0.7286
##       Balanced Accuracy : 0.8481
##
```

```
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```



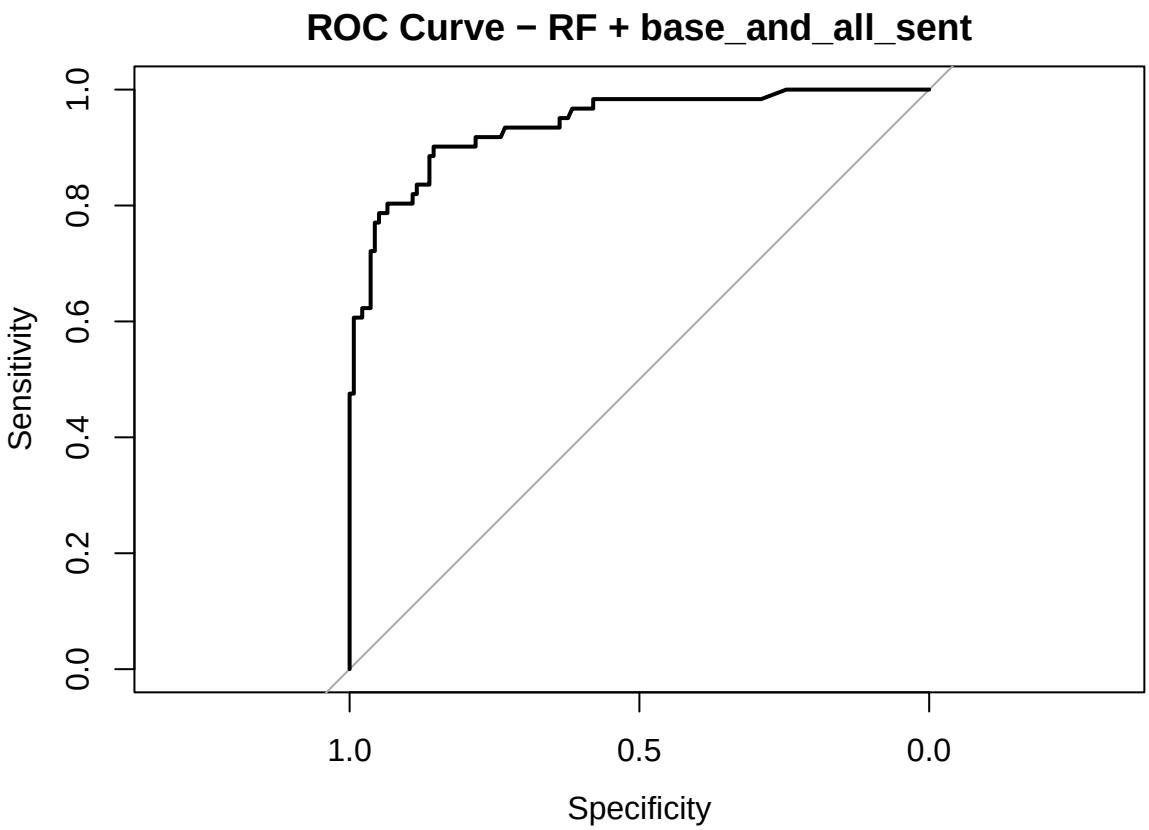

```
## RF + roberta AUC: 0.9311594
```


Mean Decrease Accuracy − RF + base_and_all_s
rating_overall
flags_business_outlook
sent_roberta_avg
rating_culture_values
rating_senior_leadership
sent_vader_avg
sent_distilbert_avg
rating_work_life
rating_career_opportunities
rating_diversity_inclusion
flags_ceo_approval
rating_compensation_benefits
sent_distilbert_sd
career_missing
sent_roberta_sd
sector
sent_vader_sd
employee_job_quit
covid_related_review
employee_length
0
5
10
15
20
25
30
MeanDecreaseAccuracy




```
## Best threshold for RF + base_and_all_sent : 0.4
## Confusion Matrix and Statistics
##
##           Reference
## Prediction NEGATIVE POSITIVE
##   NEGATIVE      129       12
##   POSITIVE        9       49
##
##                Accuracy : 0.8945
##                  95% CI : (0.8432, 0.9335)
##     No Information Rate : 0.6935
##     P-Value [Acc > NIR] : 1.388e-11
##
##                   Kappa : 0.7483
##
##  Mcnemar's Test P-Value : 0.6625
##
##             Sensitivity : 0.9348
##             Specificity : 0.8033
##          Pos Pred Value : 0.9149
##          Neg Pred Value : 0.8448
##              Prevalence : 0.6935
##          Detection Rate : 0.6482
##    Detection Prevalence : 0.7085
##       Balanced Accuracy : 0.8690
##
```

```
##        'Positive' Class : NEGATIVE
##

## Setting levels: control = NEGATIVE, case = POSITIVE
## Setting direction: controls < cases
```




```
## RF + base_and_all_sent AUC: 0.9392967
```

RF Model Comparison (Base features vs Sentiment Add-Ons)

```r
rf_summary <- data.frame(
  Features = names(rf_results),
  Model = "Random Forest",
  Best_mtry = sapply(rf_results, function(x) x$best_mtry),
  AUC = sapply(rf_results, function(x) as.numeric(x$eval$auc)),
  F1 = sapply(rf_results, function(x) max(x$eval$f1_results$F1)),
  Accuracy = sapply(rf_results, function(x) as.numeric(x$eval$confusion_matrix$overall["Accuracy"])),
  Best_Threshold = sapply(rf_results, function(x) x$eval$best_threshold), row.names = NULL
)

rf_summary
```

```
##            Features         Model Best_mtry       AUC        F1  Accuracy
## 1         base_only Random Forest         6 0.9308030 0.8205128 0.8944724
## 2             vader Random Forest         9 0.9384652 0.8245614 0.8994975
## 3        distilbert Random Forest         5 0.9312782 0.8103448 0.8894472
## 4           roberta Random Forest         5 0.9311594 0.8000000 0.8844221
## 5 base_and_all_sent Random Forest         3 0.9392967 0.8235294 0.8944724
##   Best_Threshold
## 1           0.45
## 2           0.55
## 3           0.45
## 4           0.50
## 5           0.40
```

Sentiment features do not boost performance as much as they did in Logistic Regression models. The gains in F1 and AUC from VADER and all sentiment features are marginal for an already great performing baseline.

Ranking every model so far (LogReg + Random Forest)

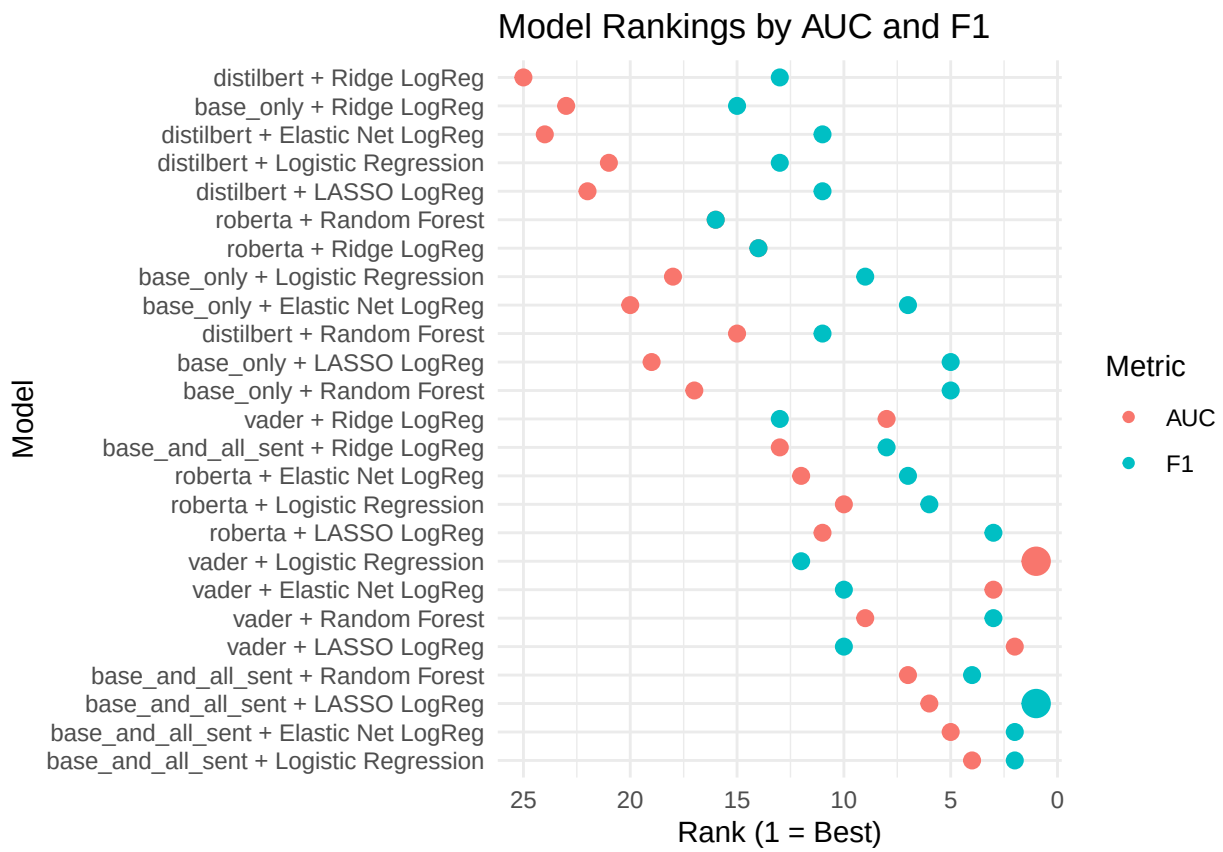
```r
logistic_df <- model_comparison %>%
  select(Features, Model, AUC, F1, Accuracy)
rf_df <- rf_summary %>% select(Features, Model, AUC, F1, Accuracy)

full_summary_df <- bind_rows(logistic_df, rf_df)

full_rank_df <- full_summary_df %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1)),
    Accuracy_Rank = dense_rank(desc(Accuracy))
  )
full_rank_df
```

```
##             Features               Model       AUC        F1  Accuracy
## 1          base_only Logistic Regression 0.9303873 0.8135593 0.8894472
## 2          base_only        LASSO LogReg 0.9300309 0.8205128 0.8944724
## 3          base_only        Ridge LogReg 0.9243288 0.8000000 0.8743719
## 4          base_only  Elastic Net LogReg 0.9294369 0.8166667 0.8894472
## 5              vader Logistic Regression 0.9476123 0.8073394 0.8944724
## 6              vader        LASSO LogReg 0.9448800 0.8113208 0.8994975
## 7              vader        Ridge LogReg 0.9389404 0.8070175 0.8894472
## 8              vader  Elastic Net LogReg 0.9442861 0.8113208 0.8994975
## 9         distilbert Logistic Regression 0.9246852 0.8070175 0.8894472
## 10        distilbert        LASSO LogReg 0.9245664 0.8103448 0.8894472
## 11        distilbert        Ridge LogReg 0.9179140 0.8070175 0.8894472
## 12        distilbert  Elastic Net LogReg 0.9242100 0.8103448 0.8894472
## 13           roberta Logistic Regression 0.9378712 0.8173913 0.8944724
## 14           roberta        LASSO LogReg 0.9369209 0.8245614 0.8994975
## 15           roberta        Ridge LogReg 0.9327631 0.8032787 0.8793970
## 16           roberta  Elastic Net LogReg 0.9360893 0.8166667 0.8894472
## 17 base_and_all_sent Logistic Regression 0.9428605 0.8318584 0.9045226
```

```
## 18 base_and_all_sent        LASSO LogReg 0.9417914 0.8392857 0.9095477
## 19 base_and_all_sent        Ridge LogReg 0.9351390 0.8141593 0.8944724
## 20 base_and_all_sent  Elastic Net LogReg 0.9421478 0.8318584 0.9045226
## 21         base_only       Random Forest 0.9308030 0.8205128 0.8944724
## 22             vader       Random Forest 0.9384652 0.8245614 0.8994975
## 23        distilbert       Random Forest 0.9312782 0.8103448 0.8894472
## 24           roberta       Random Forest 0.9311594 0.8000000 0.8844221
## 25 base_and_all_sent       Random Forest 0.9392967 0.8235294 0.8944724
##                                Model_Label AUC_Rank F1_Rank Accuracy_Rank
## 1          base_only + Logistic Regression       18       9             5
## 2                 base_only + LASSO LogReg       19       5             4
## 3                 base_only + Ridge LogReg       23      15             8
## 4           base_only + Elastic Net LogReg       20       7             5
## 5              vader + Logistic Regression        1      12             4
## 6                     vader + LASSO LogReg        2      10             3
## 7                     vader + Ridge LogReg        8      13             5
## 8               vader + Elastic Net LogReg        3      10             3
## 9         distilbert + Logistic Regression       21      13             5
## 10               distilbert + LASSO LogReg       22      11             5
## 11               distilbert + Ridge LogReg       25      13             5
## 12         distilbert + Elastic Net LogReg       24      11             5
## 13           roberta + Logistic Regression       10       6             4
## 14                  roberta + LASSO LogReg       11       3             3
## 15                  roberta + Ridge LogReg       14      14             7
## 16            roberta + Elastic Net LogReg       12       7             5
## 17 base_and_all_sent + Logistic Regression        4       2             2
## 18        base_and_all_sent + LASSO LogReg        6       1             1
## 19        base_and_all_sent + Ridge LogReg       13       8             4
## 20  base_and_all_sent + Elastic Net LogReg        5       2             2
## 21               base_only + Random Forest       17       5             4
## 22                   vader + Random Forest        9       3             3
## 23              distilbert + Random Forest       15      11             5
## 24                 roberta + Random Forest       16      16             6
## 25       base_and_all_sent + Random Forest        7       4             4
```

```
rank_plot_data <- full_summary_df %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1))
  ) %>%
  select(Model_Label, AUC_Rank, F1_Rank) %>%
  pivot_longer(
    cols = c(AUC_Rank, F1_Rank),
    names_to = "Metric",
    values_to = "Rank"
  ) %>%
  mutate(
    Metric = recode(Metric,
      AUC_Rank = "AUC",
      F1_Rank = "F1"
    )
  )
```

```r
ggplot(rank_plot_data, aes(x = reorder(Model_Label, Rank), y = Rank, color = Metric)) +
  geom_point(aes(size = Rank == 1)) +
  coord_flip() +
  scale_y_reverse() +
  scale_size_manual(values = c("FALSE" = 2.5, "TRUE" = 4.5), guide = "none") +
  theme_minimal() +
  labs(
    title = "Model Rankings by AUC and F1",
    x = "Model",
    y = "Rank (1 = Best)"
  )
```



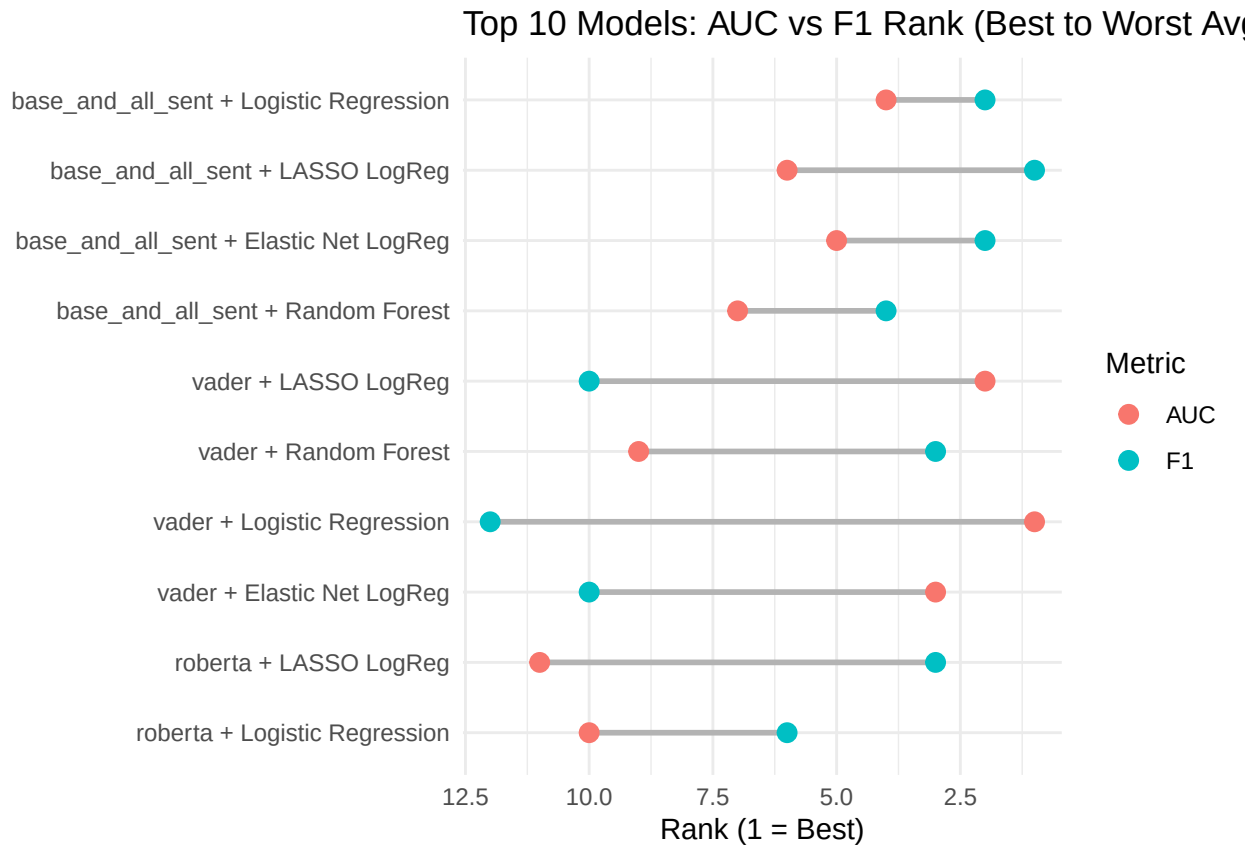
Let's get average ranks (using AUC and F1 ranks) and select the best 10 models overall

```r
full_rank_df <- full_summary_df %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1)),
    Avg_Rank = (AUC_Rank + F1_Rank) / 2
  )
```

```r
top_models <- full_rank_df %>%
  slice_min(Avg_Rank, n = 10) %>%
  arrange(Avg_Rank)

top_models %>% select(Model_Label, AUC_Rank, F1_Rank, Avg_Rank)
```

```
##                                Model_Label AUC_Rank F1_Rank Avg_Rank
## 1  base_and_all_sent + Logistic Regression        4       2      3.0
## 2         base_and_all_sent + LASSO LogReg        6       1      3.5
## 3   base_and_all_sent + Elastic Net LogReg        5       2      3.5
## 4        base_and_all_sent + Random Forest        7       4      5.5
## 5                     vader + LASSO LogReg        2      10      6.0
## 6                    vader + Random Forest        9       3      6.0
## 7              vader + Logistic Regression        1      12      6.5
## 8               vader + Elastic Net LogReg        3      10      6.5
## 9                   roberta + LASSO LogReg       11       3      7.0
## 10           roberta + Logistic Regression       10       6      8.0
```

```r
top_models <- top_models %>%
  arrange(Avg_Rank) %>%
  mutate(Model_Label = factor(Model_Label, levels = rev(Model_Label)))

ggplot(top_models, aes(y = Model_Label)) +
  geom_segment(
    aes(x = AUC_Rank, xend = F1_Rank, yend = Model_Label),
    color = "gray70",
    linewidth = 1
  ) +
  geom_point(aes(x = AUC_Rank, color = "AUC"), size = 3) +
  geom_point(aes(x = F1_Rank, color = "F1"), size = 3) +
  scale_x_reverse() +
  theme_minimal() +
  labs(
    title = "Top 10 Models: AUC vs F1 Rank (Best to Worst Avg Rank)",
    x = "Rank (1 = Best)",
    y = NULL,
    color = "Metric"
  )
```



To compare the top-performing models, we examine their rankings in both AUC and F1 score, focusing on the ten best models overall. A clear pattern emerges: models that incorporate all sentiment features consistently perform the best. In particular, LASSO and Elastic Net logistic regression achieve the strongest F1 rankings while maintaining solid AUC performance, indicating they strike a good balance between classification accuracy and ranking ability.

In contrast, models using only VADER sentiment tend to rank higher in AUC but lower in F1. This suggests they are better at ordering observations correctly but less effective at making accurate classifications at a fixed threshold.

Random Forest models fall in between—generally competitive, but they do not surpass the best-performing regularized logistic models.

Overall, the results suggest that combining richer sentiment features with regularization leads to the most reliable performance, especially when both ranking quality (AUC) and classification accuracy (F1) are considered together.

Filtering models that used non-sentiment base features

```
rank_plot_data_base <- full_summary_df %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1)),
    Avg_Rank = (AUC_Rank + F1_Rank) / 2
  ) %>%
```

```r
  filter(Features == "base_only")

rank_plot_data_base %>%
  select(Model_Label, AUC_Rank, F1_Rank, Avg_Rank) %>%
  arrange(Avg_Rank)
```

```
##                        Model_Label AUC_Rank F1_Rank Avg_Rank
## 1        base_only + Random Forest       17       5     11.0
## 2         base_only + LASSO LogReg       19       5     12.0
## 3 base_only + Logistic Regression       18       9     13.5
## 4  base_only + Elastic Net LogReg       20       7     13.5
## 5         base_only + Ridge LogReg       23      15     19.0
```

Plot best models for base features

```r
base_only_models <- full_summary_df %>%
  mutate(
    Model_Label = paste(Features, Model, sep = " + "),
    AUC_Rank = dense_rank(desc(AUC)),
    F1_Rank = dense_rank(desc(F1)),
    Avg_Rank = (AUC_Rank + F1_Rank) / 2
  ) %>%
  filter(Features == "base_only") %>%
  arrange(Avg_Rank) %>%
  mutate(Model_Label = factor(Model_Label, levels = rev(Model_Label)))

ggplot(base_only_models, aes(y = Model_Label)) +
  geom_segment(
    aes(x = AUC_Rank, xend = F1_Rank, yend = Model_Label),
    color = "gray70",
    linewidth = 1
  ) +
  geom_point(aes(x = AUC_Rank, color = "AUC"), size = 3) +
  geom_point(aes(x = F1_Rank, color = "F1"), size = 3) +
  scale_x_reverse() +
  theme_minimal() +
  labs(
    title = "Base-Only Models: AUC vs F1 Rank (Best to Worst Avg Rank)",
    x = "Rank (1 = Best)",
    y = NULL,
    color = "Metric"
  )
```

Among the base-only models, Random Forest achieves the best overall ranking when evaluated across all models, driven by a strong F1 rank despite a weaker AUC. This suggests that while linear models such as LASSO and Logistic Regression provide slightly better discrimination, Random Forest offers a better balance between precision and recall.

Overall, however, all base-only models rank in the middle compared to sentiment-enhanced models, indicating that incorporating sentiment features provides a meaningful improvement in performance.